Implementation and comparison of shortest common superstring algorithms

Adam Burszka

Bachelor's Thesis
Supervisor: D.Sc. Krzysztof Turowski

Uniwersytet Jagielloński
Instytut Informatyki Analitycznej
Kraków 2026

Contents

Chapter 1

Introduction

1.1 Definitions and Problem Statement

1.1.1 Definitions

In this section, we introduce the basic concepts that will be used throughout this paper. Let Σ denote an alphabet, which is a finite set of characters. A string s is a sequence of characters derived from the alphabet Σ. The length of a string s is denoted as $|s|$.

Definition 1 (Prefix and Suffix [CR94]). *A prefix of a string $s = a_1a_2 \dots a_n$ of length j is a string denoted by $s[: j+1] = a_1a_2 \dots a_j$. Similarly, a suffix is a string denoted by $s[i :] = a_i \dots a_n$.*

Definition 2 (Substring [CR94]). *A string x is a substring of a string s if there exist strings u and v (which may be empty) such that $s = uxv$. A set of strings $S = \{s_1, s_2, \dots, s_m\}$ is considered substring-free if no string $s_i \in S$ is a substring of any other string $s_j \in S$.*

Definition 3 (Overlap, Prefix, and Distance [Blu+94]). *For two strings s and t, let y be the longest string such that $s = xy$ and $t = yz$ for some non-empty strings x and z. We define the following concepts:*

- *The **overlap** between s and t, denoted as $ov(s,t)$, is the length of the string y, i.e., $ov(s,t) = |y|$. This notion can be naturally extended to cases where t is a semi-infinite string, provided that s remains finite.*
- *The **prefix** of s with respect to t, denoted as $pref(s,t)$, is defined as the string x. Therefore, s can be uniquely decomposed as $s = pref(s,t)y$.*
- *The **distance** from s to t, denoted as $d(s,t)$, is defined as the length of this prefix, meaning $d(s,t) = |pref(s,t)| = |x|$.*

Definition 4 (Common Superstring [Blu+94]). *A common superstring for a set of strings $S = \{s_1, s_2, \dots, s_m\}$ is a string s such that every $s_i \in S$ is a substring of s.*

Definition 5 (Superstring of a Sequence [Blu+94]). *For a given ordered list of strings $s_{i_1}, \ldots, s_{i_r}$, the shortest common superstring representing them, denoted as $\langle s_{i_1}, \ldots, s_{i_r} \rangle$, is defined as the concatenation of the prefixes and the final string:*

$$\langle s_{i_1}, \ldots, s_{i_r} \rangle = pref(s_{i_1}, s_{i_2}) pref(s_{i_2}, s_{i_3}) \ldots pref(s_{i_{r-1}}, s_{i_r}) s_{i_r} \tag{1.1}$$

Any optimal common superstring for a finite set of strings $S = \{s_1, \ldots, s_m\}$ always corresponds to a sequential construction of the form $\langle s_{i_1}, \ldots, s_{i_m} \rangle$ for some optimal permutation of indices $i_1, \ldots, i_m$ from the set $\{1, \ldots, m\}$. The length of such a shortest superstring is denoted as $opt(S)$.

We define $maxov(S)$ as the maximum total overlap between adjacent elements in the superstring. Formally, it is the maximum possible sum of overlaps $\sum_{j=1}^{m-1} ov(s_{i_j}, s_{i_{j+1}})$ achieved over all permutations of the input sequence. The size of the optimal solution, denoted as $opt(S)$, is strictly related to this value by the equation:

$$opt(S) = \sum_{s_i \in S} |s_i| - maxov(S) \tag{1.2}$$

From the above identity, it follows that the problem of minimizing the superstring length $opt(S)$ is equivalent to the problem of maximizing the sum of overlaps $maxov(S)$.

1.1.2 Problem Statement and Applications

Problem 1 (Shortest Common Superstring). *Given a set of strings $S = \{s_1, s_2, \ldots, s_m\}$ over a finite alphabet Σ, the problem is to find a common superstring for the set S of the minimum possible length. A string s is considered a superstring of S if every string $s_i \in S$ appears as a contiguous substring (a consecutive block) of s. The length of this optimal shortest superstring is denoted as $opt(S)$.*

Since the Shortest Common Superstring problem is NP-hard [GMS80], no polynomial-time algorithm for finding an exact solution is known, and it is widely believed that none exists (assuming P $\neq$ NP) [GJ79]. Consequently, all known exact algorithms require exponential time in the worst case, making them impractical for large problem instances encountered in practice. Therefore, research has focused on the development of polynomial-time approximation algorithms that produce near-optimal solutions with provable approximation guarantees [Vaz01].
The SCS problem has numerous real-world applications across multiple scientific and engineering fields:

- **Bioinformatics:** One of the most prominent applications of the Shortest Common Superstring problem is DNA sequence assembly [SM97]. In shotgun sequencing, long DNA molecules—often consisting of millions of base pairs—are fragmented into thousands of short, overlapping reads, typically around 500 bases long. The original DNA sequence is then reconstructed by finding a short superstring that contains all reads, making SCS a natural computational model for this task [PSU84].

- **Data Compression:** The SCS serves as a theoretical foundation for dictionary-based data compression [SS82]. A common superstring can be viewed as a maximally compressed form of a set of keywords, drastically decreasing data redundancy.
- **Operations Research:** Beyond strings and text, SCS solutions can be abstracted to solve scheduling problems, specifically in routing or scheduling operations on machines that require coordinated starting times [Law+85].

Table 1.1: History of approximation-ratio improvements for the Shortest Common Superstring (SCS) problem. Rows in **bold** indicate the algorithm(s) discussed in this work.

Year	Authors	Algorithm / Main Approach	Approximation Ratio
1994	Blum et al. [Blu+94]	Cycle Cover / Modified Greedy	3
1997	Teng, Yao [TY97]	Optimal Cycle Covers / Matchings	$2\frac{8}{9} \approx 2.89$
1997	Czumaj et al. [Czu+97]	Prefix Graph / Cycle Covers	$2\frac{5}{6} \approx 2.83$
1998	Armen, Stein [AS98]	Graph Matching Frameworks	$2\frac{2}{3} \approx 2.67$
1997	**Breslauer, Jiang, Jiang [BJJ97]**	**String Periodicity / Overlap Rotation (Algorithm 1)**	$\mathbf{2\frac{2}{3} \approx 2.67}$
1997	**Breslauer, Jiang, Jiang [BJJ97]**	**String Periodicity / Overlap Rotation (Algorithm 2)**	$\mathbf{2\frac{25}{42} \approx 2.596}$
2005	Kaplan et al. [Kap+05]	ATSP Cycle Cover Reduction	$2\frac{1}{2} = 2.50$
2013	Mucha [Muc13]	ATSP Bounding via Lyndon Words	$2\frac{11}{23} \approx 2.478$
2023	Englert et al. [EMV23]	Improved Cycle Overlap Bounds	$\frac{\sqrt{67}+14}{9} \approx 2.466$

1.2 Distance Graph and Cycle Covers

It turns out that there is a relation between the Shortest Common Superstring problem and certain graph problems, which forms the basis of the approximation algorithms discussed in this work.

Definition 6 (Traveling Salesperson Problem (TSP) [Kar72])**.** *Given a complete directed weighted graph $G = (V, E, w)$, the Traveling Salesperson Problem (TSP) asks for a minimum weight Hamiltonian cycle in G, i.e., a minimum weight cycle visiting every vertex of G exactly once.*

Definition 7 (Distance Graph [Vaz01])**.** *The distance graph for a set of strings $S = \{s_1, \ldots, s_m\}$ is a directed weighted graph $G_S = (V, E, w)$, in which the set of vertices is $V = S$, the set of edges is $E = \{(s_i, s_j) \colon 1 \leq i \neq j \leq m\}$, and the weight function assigns to each edge the distance between the corresponding strings, i.e., $w(s_i, s_j) = d(s_i, s_j)$.*

If we denote the cost of a minimum weight Hamiltonian cycle on G_S as $TSP(G_S)$, it provides a very good approximation of the shortest superstring. Specifically, for any $s_i \in S$, the relationship $TSP(G_S) \leq opt(S) \leq TSP(G_S) + |s_i|$ holds. However, since the TSP is NP-hard [Kar72], approximation algorithms utilize a relaxed version of this problem known as the cycle cover problem (also referred to as the assignment problem).

Definition 8 (Cycle Cover [Vaz01])**.** *A cycle cover in a directed weighted graph is a set of simple cycles such that every vertex of the graph is contained in exactly one of them. The weight of the cover is the total sum of the weights of all cycles comprising it.*

Unlike the TSP, a minimum weight cycle cover on any directed weighted graph can be efficiently computed in polynomial time. Specifically, it can be solved in $O(n^3)$ time using the Hungarian algorithm [Kuh55; EK72; Tom71].

Let $CYC(G_S)$ denote the weight of a minimum weight cycle cover of G_S. This weight provides a useful lower bound for the length of the optimal superstring, establishing the inequality:

$$CYC(G_S) \leq TSP(G_S) \leq opt(S) \tag{1.3}$$

However, because there is no obvious general upper bound on $opt(S)$ expressed solely in terms of $CYC(G_S)$, designing effective approximation algorithms relies on specific properties of the strings. In fact, on general distance graphs the gap between $CYC(G_S)$ and $TSP(G_S)$ can be made arbitrarily large [SG76].

Chapter 2

Breslauer-Jiang-Jiang (1996) Algorithms

In this chapter, we present the shortest superstring approximation algorithms of Breslauer, Jiang, and Jiang [BJJ97]. We first describe the generic approximation algorithm, then establish the overlap-rotation lemma, the main theorem on which the approximation guarantees of both algorithms rely, and finally present the two algorithms themselves, achieving approximation ratios of $2\frac{2}{3}$ and $2\frac{25}{42} \approx 2.596$, respectively. Let us first introduce necessary definitions.

2.1 Preliminaries

Definition 9 (Factor and Periodicity [BJJ97]). *A string x is a factor of a string s if $s = x^i y$ for some positive integer i and a prefix y of x (where y may be empty). The shortest such factor of a non-empty string s, denoted as $factor(s)$, defines the period of s, which is denoted as $period(s) = |factor(s)|$.*

Definition 10 (Semi-Infinite Strings and Equivalence [BJJ97]). *A semi-infinite string $s = a_1 a_2 \ldots$ is said to be periodic if $s = xs$ for some non-empty string x. The shortest such x is called the factor of s. Two (periodic semi-infinite) strings s and t are equivalent if their factors are cyclic shifts of each other, i.e., if there are strings x and y such that $factor(s) = xy$ and $factor(t) = yx$. Otherwise, they are inequivalent.*

Example 1. *Let $s = (ab)^\infty = abababab\ldots$ and $t = (ba)^\infty = bababababa\ldots$. Then $factor(s) = ab$ and $factor(t) = ba$, and taking $x = a$, $y = b$ gives $factor(s) = xy$ and $factor(t) = yx$. Hence s and t are equivalent.*

In contrast, let $s' = (aabb)^\infty = aabbaabb\ldots$ and $t' = (abab)^\infty = abababab\ldots$. Since $factor(t') = abab$ is not a cyclic shift of $factor(s') = aabb$, the strings s' and t' are inequivalent.

For each string s, let s^i denote the concatenation of i consecutive copies of s. Furthermore, let s^∞ denote the semi-infinite string $sss\ldots$, and let $s_\infty = factor(s)^\infty$ denote the periodic semi-infinite string that begins with s. Note that, in general, $s^\infty \neq s_\infty$.

Example 2. *Let $s = aaba$. Then $factor(s) = aab$, and we have*

$$s^\infty = aabaaaba\ldots \qquad while \qquad s_\infty = factor(s)^\infty = aabaabaab\ldots,$$

giving that $s^\infty \neq s_\infty$.

We extend the notion of equivalence from Definition 10 to finite strings: a finite string s is equivalent to a (periodic semi-infinite) string t if s_∞ and t are equivalent, and two finite strings s and s' are equivalent if s_∞ and s'_∞ are equivalent, i.e., if $factor(s)$ and $factor(s')$ are cyclic shifts of each other. In particular, every string s is equivalent to s_∞.

Lemma 1. *Suppose that a string x is contained in a string y, and y is contained in a string z. If x and z are equivalent, then y is also equivalent to x and z.*

2.1.1 Cycles and Periodicity

The following facts establish the connection between a cycle in the distance graph G_S and the periodicity of the strings obtained by breaking that cycle. Let $C = s_{i_1}, \ldots, s_{i_r}, s_{i_1}$ be a cycle in G_S. Without loss of generality, assume that C has the minimum weight among all cycles in G_S containing $s_{i_1}, \ldots, s_{i_r}$. Let $w(C)$ denote the weight of this cycle. We define $\langle s_{i_1}, \ldots, s_{i_r}\rangle$ as the string obtained by breaking the cycle C at s_{i_r}.

Fact 1. *The string $\langle s_{i_1}, \ldots, s_{i_r}\rangle$ is a substring of $factor(\langle s_{i_1}, \ldots, s_{i_r}\rangle)s_{i_1} = \langle s_{i_1}, \ldots, s_{i_r}, s_{i_1}\rangle$.*

Fact 2. *The factor of the broken cycle string can be expressed as:*

$$factor(\langle s_{i_1}, \ldots, s_{i_r}\rangle) = pref(s_{i_1}, s_{i_2}) \ldots pref(s_{i_{r-1}}, s_{i_r}) pref(s_{i_r}, s_{i_1}) \tag{2.1}$$

From Lemma 2, three important corollaries follow immediately:

Corollary 3. *The weight of the cycle equals the period of the generated superstring:*

$$w(C) = d(s_{i_1}, s_{i_2}) + \cdots + d(s_{i_{r-1}}, s_{i_r}) + d(s_{i_r}, s_{i_1}) = period(\langle s_{i_1}, \ldots, s_{i_r}\rangle) \tag{2.2}$$

Corollary 4. *The strings generated by different breaking points of the cycle, namely $\langle s_{i_1}, \ldots, s_{i_r}\rangle, \ldots, \langle s_{i_r}, s_{i_1}, \ldots, s_{i_{r-1}}\rangle$, are all mutually equivalent.*

Corollary 5. *It holds that $factor(\langle s_{i_1}, \ldots, s_{i_r}, s_{i_1}\rangle) = factor(\langle s_{i_1}, \ldots, s_{i_r}\rangle)$. That is, the string $\langle s_{i_1}, \ldots, s_{i_r}, s_{i_1}\rangle$ is equivalent to $\langle s_{i_1}, \ldots, s_{i_r}\rangle$.*

It is important to note that, in general, the individual strings $s_{i_1}, \ldots, s_{i_r}$ are neither mutually equivalent, nor is s_{i_1} equivalent to the combined string $\langle s_{i_1}, \ldots, s_{i_r}\rangle$.

Finally, if two cycles generate equivalent strings, they can be merged without increasing the cycle weight:

Lemma 2. *Let $C = s_{i_1}, \ldots, s_{i_r}, s_{i_1}$ and $C' = s_{j_1}, \ldots, s_{j_l}, s_{j_1}$ be two distinct cycles. If the string $\langle s_{i_1}, \ldots, s_{i_r}\rangle$ is equivalent to $\langle s_{j_1}, \ldots, s_{j_l}\rangle$, then there exists a third cycle $\tilde{C}$ with weight $w(C)$ containing all vertices from both C and C'.*

2.2 The Generic Approximation Algorithm

Many approximation algorithms for the shortest superstring problem share a similar structural framework based on cycle covers. In this section, we outline a generic approach that serves as a foundation for more advanced algorithms. We will also demonstrate that this generic algorithm, originally proposed by Blum et al. [Blu+94], achieves an approximation ratio of 3, which acts as a theoretical warm-up before introducing more complex overlap bounds.

2.2.1 Algorithm Outline

The generic approximation algorithm constructs a superstring by computing optimal cycle covers on the distance graph, generating representative strings for each cycle, and then repeating the process to merge these representatives. The exact steps are:

1. **Initial Distance Graph:** Construct the distance graph G_S for the input set of strings S.
2. **First Cycle Cover:** Find a minimum weight cycle cover $\mathcal{C}_S$ on the graph G_S.
3. **Cycle Representatives:** For each cycle $C = s_{i_1}, \ldots, s_{i_r}, s_{i_1} \in \mathcal{C}_S$, choose a representative string t_C such that for some index j:
   (i) t_C contains the string $\langle s_{i_{j+1}}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_j} \rangle$, and
   (ii) t_C is contained in the string $\langle s_{i_j}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_{j-1}}, s_{i_j} \rangle$.
4. **Second Distance Graph:** Let T be the set of all chosen representative strings t_C. Construct the distance graph G_T for the set T.
5. **Second Cycle Cover:** Find a minimum weight cycle cover $\mathcal{C}_T$ on the graph G_T.
6. **Breaking Cycles:** Break each cycle of $\mathcal{C}_T$ arbitrarily to obtain a superstring of the elements contained in that cycle.
7. **Final Concatenation:** Arbitrarily concatenate the strings obtained in Step 6 to produce the final superstring $\tilde{s}$ for the set S.

A key distinction between this algorithm and earlier approaches lies in Step 3. Instead of simply picking one of the original strings contained in the cycle C, the algorithm selects a string t_C that acts as a short superstring of the elements in C, without being excessively long.

2.2.2 Approximation Ratio Analysis

To prove that this generic algorithm achieves a 3-approximation, we rely on bounds on the optimal superstring length of the representative set T and on the maximum possible overlap between inequivalent strings.

Lemma 3. *The length of the optimal superstring for the set T is bounded by:*

$$opt(T) \leq opt(S) + CYC(G_S) \leq 2opt(S) \tag{2.3}$$

Proof. Second inequality is given by equation (1.3). First inequality holds because each $t_C \in T$ is contained in $\langle s_{i_j}, \ldots, s_{i_{j-1}}, s_{i_j} \rangle$. So, we can take an optimal superstring of S, and for each $t_C \in T$, substitute the occurrence of s_{i_j} with $\langle s_{i_j}, \ldots, s_{i_{j-1}}, s_{i_j} \rangle$. Each such substitution increases the length by $w(C)$, since we replace $|s_{i_j}|$ with $w(C) + |s_{i_j}|$, so in total we add $CYC(G_S)$ to the length. Since each t_C is a substring of the resulting string, it is a superstring of T of length $opt(S) + CYC(G_S)$, which gives the first inequality. □

Since the weight of the minimum cycle cover on G_T cannot exceed the length of the optimal superstring for T, Lemma 3 immediately implies that $CYC(G_T) \leq opt(T) \leq 2opt(S)$.

The amount of overlap lost when breaking the cycles in Step 6 depends on the periodicity of the strings in T. This is bounded by the following property of discrete periodic functions:

Lemma 4 ([Blu+94])**.** *For any two inequivalent strings s and t, their maximum overlap is strictly bounded by the sum of their periods:*

$$ov(s,t) \leq period(s) + period(t) \tag{2.4}$$

Proof. Let $p = period(s)$, $q = period(t)$, and suppose for contradiction that $u = ov(s,t)$ satisfies $|u| \geq p+q$. Write $u_{k,l}$ for the substring of u from position k to l (inclusive). Since u is a suffix of s, and s is a substring of $factor(s)^\infty$, u inherits period p: $u_{k,l} = u_{k+p,l+p}$ whenever both sides are defined. Symmetrically, since u is a prefix of t, u inherits period q. Moreover, because u is literally t's own prefix of length $|u| \geq q$ we have $u_{1,q} = factor(t)$.

Case $p = q$. Then $u_{1,p} = factor(t)$ as noted above. On the other hand, u is a substring of $factor(s)^\infty$, so $u_{1,p}$ is a substring of length $p = period(s)$ of that periodic string, i.e., a cyclic rotation of $factor(s)$. Hence $factor(t)$ is a cyclic rotation of $factor(s)$, so s and t are equivalent – a contradiction.

Case $p \neq q$, say without loss of generality $p > q$. Combining the two inherited periods of u:

$$x \;=\; u_{1,p} \;=\; u_{1+q,\,p+q} \;=\; u_{1+q,\,p} \cdot u_{p+1,\,p+q} \;=\; u_{1+q,\,p} \cdot u_{1,\,q} \tag{2.5}$$

(the first equality uses u's period q, the second is just splitting the range, the third uses u's period p). Writing $a = u_{1,q} = factor(t)$ and $b = u_{1+q,p}$ (so $|a| = q$, $|b| = p - q$), this reads $x = b \cdot a$. But trivially $x = u_{1,p} = a' \cdot b'$ for $a' = u_{1,q} = a$ and $b' = u_{1+q,p} = b$ as well, i.e. $x = a \cdot b$. So $x = ab = ba$ with a, b both non-empty.

Since $x = u_{1,p}$ is a cyclic rotation of $factor(s)$, and $ab = ba$ makes x a power of a shorter string z [LS62], it follows that $factor(s)$ is also a power of a shorter string, which contradicts the fact that $factor(s)$ is minimal.

In both cases we reach a contradiction, so $ov(s,t) = |u| < p+q = period(s) + period(t)$. □

Based on the structural properties of superstrings, the string t_C selected in Step 3 is equivalent to the cyclic superstring $\langle s_{i_1}, \ldots, s_{i_r} \rangle$. Because the initial cycle cover $\mathcal{C}_S$ has minimum weight, the strings in the set T are guaranteed to be mutually inequivalent. Therefore, Lemma 4 applies to all strings in T.

Let OV denote the total overlap lost by breaking the edges in Step 6. Since the sum of the overlaps between adjacent strings in T cannot exceed the sum of their periods, OV is bounded by the total weight of the initial cycle cover. By the relationship between cycle weight and period length established in Corollary 3, we have:

$$OV \leq \sum_{C \in \mathcal{C}_S} w(C) = CYC(G_S) \tag{2.6}$$

By combining these observations, we can establish the upper bound on the length of the final superstring $\tilde{s}$ produced by the algorithm:

$$|\tilde{s}| = CYC(G_T) + OV \leq CYC(G_T) + CYC(G_S) \leq 2opt(S) + opt(S) = 3opt(S) \tag{2.7}$$

Thus, the generic algorithm safely achieves an approximation ratio of 3.

2.3 The Overlap-Rotation Lemma

To establish tighter bounds on the overlap between strings, it is necessary to explore the combinatorial properties of periodic strings. The foundation of these properties relies on the concepts of unbordered strings and factorizations.

2.3.1 Combinatorial Properties of Strings

Definition 11 (Unbordered String [Lot83]). *A string w is unbordered if it has no proper prefix that is also a suffix. In other words $ov(w, w) = 0$ and $factor(w) = w$.*

Definition 12 (Local Factor and Critical Factorization [Lot83]). *Given a non-trivial factorization $w = uv$ (a partition of w into a non-empty prefix u and a non-empty suffix v), a non-empty string z is a repetition at (u, v) if z is suffix-comparable with u (i.e. z is a suffix of u, or u is a suffix of z) and prefix-comparable with v (i.e. z is a prefix of v, or v is a prefix of z). The local factor of the factorization (u, v) is the shortest repetition at (u, v).*

A non-trivial factorization $w = uv$ is called a critical factorization if its local factor has the exact same length as $period(w)$.

The concept of critical factorization applies to both finite and infinite strings, and its existence is guaranteed by the following theorem:

Theorem 6 (Critical Factorization Theorem [CV78; Lot83]). *Given any $period(w) - 1$ consecutive non-trivial factorizations of a string w, at least one of them is a critical factorization.*

The behavior of critical factorizations under string rotation is characterized by the following useful lemma:

Lemma 5. *Let w be an unbordered string with the critical factorization $w = uv$. Then:*

1. *The rotation $w' = vu$ of w is also unbordered.*
2. *vu is a critical factorization of w'.*

Proof.

(1) Assume that $w' = vu$ is not unbordered. Then there is a string x $(0 < |x| < |w|)$ that is a proper prefix and suffix of w'. This string is then a local factor of $w = uv$, because it is consistent with both sides of the factorization. We have $|factor(w)| = |w| = period(w) \neq |x|$, which contradicts the fact that $w = uv$ is a critical factorization.

(2) Assume that $w' = vu$ is not a critical factorization. Then there is a string x $(0 < |x| < |w|)$ that is consistent with both sides of the factorization. We also know that $|x| > \min\{|u|, |v|\}$, because w is unbordered – otherwise we would have $v = tx$, $u = xk$, $w = xktx$. Without loss of generality, assume that $|v| \leq |u|$, so $|x| > |v|$ and $x = tv$. We have two cases:

   (2.1) $|x| \leq |u|$. Then $u = xk = tvk$ and $x = tv$, so tv is consistent with both sides of the factorization $w = uv = (tvk)(v)$ and $|kv| = |x| < |w|$, which contradicts the fact that $w = uv$ is a critical factorization.

   (2.2) $|x| > |u|$. Then $x = tv = uk$ with $|t| < |u|$ and $|k| < |v|$, since $|x| < |w| = |uv|$. This implies that $ov(u, v) > 0$, which contradicts the fact that w' is unbordered.

□

2.3.2 Overlap Bounds

We now prove the Overlap-Rotation Lemma, which forms the basis for the improved approximation bounds of the shortest superstring algorithms presented later. For a semi-infinite string $\alpha = a_1 a_2 \ldots$, we denote its rotation as $\alpha[k] = a_k a_{k+1} \ldots$.

Lemma 6 (Overlap-Rotation Lemma)**.** *Let α be a periodic semi-infinite string. There exists an integer k such that for any finite string s that is inequivalent to α:*

$$ov(s, \alpha[k]) < period(s) + \frac{1}{2} period(\alpha). \tag{2.8}$$

Furthermore, if $period(s) \leq period(\alpha)$, the bound is even tighter:

$$ov(s, \alpha[k]) < \frac{2}{3}(period(s) + period(\alpha)). \tag{2.9}$$

Proof.

Claim 1. *There exists a suffix α' of α that has a critical factorization $\alpha' = kt$ where $|k| \leq \frac{1}{2}period(\alpha)$.*

Proof. Let $\alpha = y\alpha'$ be some critical factorization of α, and let $f = factor(\alpha')$. We claim that f is unbordered: otherwise we would have $f = xf'x$ and $period(\alpha) = period(\alpha')$, so x would be a local factor with $|x| < period(\alpha)$, a contradiction. Now take a critical factorization $f = uv$:

(1) $|u| \leq \frac{1}{2}period(\alpha)$: then $\alpha' = (uv)^\infty$.

(2) $|u| > \frac{1}{2}period(\alpha)$: then by Lemma 5, $\alpha' = (vu)^\infty$.

□

We can now proceed with the proof of Lemma 6. Let α' and k be as in Claim 1. We state that

$$ov(s, \alpha') < period(s) + |k| \leq period(s) + \frac{1}{2}period(\alpha). \tag{2.10}$$

The second inequality follows from the bound on $|k|$ given by Claim 1. We prove the first inequality by contradiction. Assume that

$$ov(s, \alpha') \geq period(s) + |k|. \tag{2.11}$$

This means there is a string x, $|x| = period(s)$, that is consistent with both sides of the factorization $\alpha' = kt$. Indeed, α' can be expressed as $\alpha' = kx\alpha''$, so x is consistent with the right side of the factorization, and it is also consistent with the left side because the whole kx is a substring of s and $|x| = period(s)$. We consider three cases:

(1) $period(s) = period(\alpha)$: then, since s and α are inequivalent,

$$ov(s, \alpha) < period(s) \tag{2.12}$$

holds, so we may skip this case.

(2) $period(s) < period(\alpha)$: this contradicts the fact that $\alpha' = kt$ is a critical factorization.

(3) $period(s) > period(\alpha)$: then x also has a factor of length $period(\alpha)$, so $x = y^i z$ where $|y| = period(\alpha)$. If $|z| = 0$, then α and s are equivalent, a contradiction. If $|z| > 0$, then z is consistent with both sides of the factorization and $|z| < |y| = period(\alpha)$, again a contradiction.

It remains to prove the second part of the lemma. We have $period(s) \leq period(\alpha)$, and also $ov(s, \alpha') < period(\alpha)$ because $factor(\alpha')$ is unbordered (see proof of Claim 1). Combining both inequalities gives the strict bound:

$$ov(s, \alpha') < \min\left\{period(s) + \frac{1}{2}period(\alpha), period(\alpha)\right\} \leq \frac{2}{3}(period(s) + period(\alpha)) \tag{2.13}$$

□

The constructive nature of this proof implies that finding this rotation requires at most two computations of critical factorizations, which can be executed in time linear to $period(\alpha)$ [CP91; CR94]. Further in work, the specific optimal rotation $\alpha[k]$ that satisfies Lemma 6 is conventionally denoted by notation $\vec{\alpha}$.

2.4 The $2\frac{2}{3}$-Approximation Algorithm

The algorithm proposed by authors improves the generic shortest superstring algorithm by refining the selection of cycle representatives before the second cycle cover is computed. The key idea is to choose representative strings so that the overlap between any two representatives is sufficiently small. As a result, the total overlap lost OV during the cycle-breaking phase is reduced, leading to a better approximation ratio.

2.4.1 Choosing Cycle Representatives

We must select a representative string t_C for each cycle that satisfies the superstring containment conditions and whose infinite periodic extension is aligned with the corresponding optimal rotation from Lemma 6.

Lemma 7. *For any cycle $C = s_{i_1}, \ldots, s_{i_r}, s_{i_1} \in \mathcal{C}_S$, there exists a string t_C such that for some index j:*

1. *t_C contains the string $\langle s_{i_{j+1}}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_j} \rangle$.*
2. *t_C is contained in the string $\langle s_{i_j}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_{j-1}}, s_{i_j} \rangle$.*
3. *$(t_C)_\infty = \overrightarrow{\langle s_{i_1}, \ldots, s_{i_r} \rangle}_\infty$.*

Proof. Order the equivalent cyclic strings $\langle s_{i_1}, \ldots, s_{i_r} \rangle, \ldots, \langle s_{i_r}, s_{i_1}, \ldots, s_{i_{r-1}} \rangle$ according to their first appearances in the infinite optimally rotated string $\overrightarrow{\langle s_{i_1}, \ldots, s_{i_r} \rangle}_\infty$. This ordering is unique.

Suppose that the string $\langle s_{i_{j+1}}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_j} \rangle$ appears first in this ordering. Let t be the prefix of $\overrightarrow{\langle s_{i_1}, \ldots, s_{i_r} \rangle}_\infty$ that ends exactly at $\langle s_{i_{j+1}}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_j} \rangle$ (inclusive). Consequently, t is naturally contained in $\langle s_{i_j}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_{j-1}}, s_{i_j} \rangle$. If it were not, the string $\langle s_{i_j}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_{j-1}} \rangle$ would have to appear before $\langle s_{i_{j+1}}, \ldots, s_{i_r}, s_{i_1}, \ldots, s_{i_j} \rangle$ in the infinite string, which fundamentally contradicts our initial choice of the earliest appearing string.

The chosen string t is designated as our representative t_C. This selection process can be executed in polynomial (and in fact, linear) time. From Lemma 6 we can get $\overrightarrow{\langle s_{i_1}, \ldots, s_{i_r} \rangle}_\infty$ in linear time to $|\langle s_{i_1}, \ldots, s_{i_r} \rangle|$. Then we only need to find which string appears first which can be easily done by looking which concatenation point appears first in rotated string.

$$\langle s_{i_1}, \ldots, s_{i_r} \rangle = pref(s_{i_1}, s_{i_2}) pref(s_{i_2}, s_{i_3}) \ldots pref(s_{i_{r-1}}, s_{i_r}) s_{i_r}$$

□

2.4.2 Algorithm Outline

With the representative strings correctly defined we can modify steps 3 and 6 of generic algorithm and obtaine improved version:

1. **Initial Distance Graph:** Construct the distance graph G_S for the input set S.
2. **First Cycle Cover:** Find a minimum weight cycle cover $\mathcal{C}_S$ on the graph G_S.
3. **Optimal Representatives:** For each cycle $C \in \mathcal{C}_S$, compute and choose the representative string t_C exactly as described in Lemma 7.
4. **Second Distance Graph:** Let T be the set of all chosen strings t_C. Construct the distance graph G_T.
5. **Second Cycle Cover:** Find a minimum weight cycle cover $\mathcal{C}_T$ on the graph G_T.
6. **Strategic Cycle Breaking:** For each cycle of $\mathcal{C}_T$, break the cycle by explicitly deleting an edge that goes from a string to a string of equal or larger period. This creates a superstring of the elements within the cycle.
7. **Final Concatenation:** Arbitrarily concatenate the resulting strings to produce the final superstring $\tilde{s}$ for the set S.

2.4.3 Approximation Ratio Analysis

It is crucial to note that small cycles in $\mathcal{C}_T$ are not given any special structural treatment. Instead, the algorithm relies entirely on the precise condition in Step 6: cutting an edge that leads to a string with an equal or larger period.

Theorem 7. *The length of the final superstring $\tilde{s}$ produced by the algorithm satisfies $|\tilde{s}| \leq 2\frac{2}{3}opt(S) \approx 2.67opt(S)$.*

Proof. Let OV denote the total overlap lost by cutting the edges during Step 6. Because the representative strings comprising the set T are mutually inequivalent, we can apply the bounds provided by the Overlap-Rotation Lemma alongside the established period-weight corollaries. Moreover, since Step 6 always cuts an edge leading to a string of equal or larger period, every such edge satisfies $period(t_C) \leq period(t_{C'})$ for the pair of representatives it connects, which is exactly the condition required to invoke the tighter $\frac{2}{3}$ bound of the Overlap-Rotation Lemma rather than the general $period(s) + period(\alpha)$ bound.

$$OV \leq \frac{2}{3} \sum_{C \in \mathcal{C}_S} period(t_C) = \frac{2}{3} \sum_{C \in \mathcal{C}_S} w(C) = \frac{2}{3} CYC(G_S) \leq \frac{2}{3} opt(S) \tag{2.14}$$

From Lemma 3 we know that the weight of the cycle cover $CYC(G_T)$ cannot exceed $2opt(S)$, we can sum these components to find the strict upper bound on the length of the final superstring $\tilde{s}$:

$$|\tilde{s}| = CYC(G_T) + OV \leq 2opt(S) + \frac{2}{3} opt(S) = 2\frac{2}{3} opt(S) \tag{2.15}$$

This concludes the proof, guaranteeing the $2\frac{2}{3}$ approximation ratio. □

2.4.4 Computational Complexity

Let $n = |S|$ and let $L = \sum_{s \in S} |s|$ denote the total input length. Steps 1 and 4 each build a distance graph on at most n vertices. Steps 2 and 5 each compute a minimum weight cycle cover using the Hungarian algorithm, which runs in $O(n^3)$ time [Kuh55; EK72; Tom71]. By Lemma 7, computing the representative t_C for a cycle C takes time linear in $|\langle s_{i_1}, \ldots, s_{i_r} \rangle|$. Since the cycles of $\mathcal{C}_S$ partition S, Step 3 takes time $O(L)$ overall. Steps 6 and 7 are also linear in the size of the strings being concatenated, so they take $O(L)$ time as well. Since the two cycle cover computations take $O(n^3)$ time and all remaining steps take $O(L)$ time, the algorithm runs in $O(n^3 + L)$ time.

2.5 The $2\frac{25}{42}$-Approximation Algorithm

The approach utilized by the $2\frac{25}{42}$-approximation algorithm is similar to earlier methods that rely on overlap maximization. The core structural steps resemble the generic algorithm, with the critical difference lying in the final step: instead of computing a second cycle cover, the algorithm employs a sophisticated overlap approximation algorithm to merge the cycle representatives.

2.5.1 Algorithm Outline

The cycle representatives t_C are chosen using the exact same optimal rotation conditions as established in the $2\frac{2}{3}$-approximation algorithm. The steps are defined as follows:

1. **Initial Distance Graph:** Construct the distance graph G_S for the set S.
2. **First Cycle Cover:** Find a minimum weight cycle cover $\mathcal{C}_S$ on the graph G_S.
3. **Optimal Representatives:** For each cycle $C \in \mathcal{C}_S$, compute and choose the representative string t_C exactly as described in Lemma 7.
4. **Overlap Approximation:** Let T be the set of all chosen strings t_C. Construct a superstring of T using a δ-approximation maximum overlap algorithm.

2.5.2 Approximation Ratio Analysis

To establish the final approximation ratio, we first need to define the relationship between the length of the produced superstring and the maximum overlap of the set T.

Lemma 8. *Let $apx(T)$ be the length of the superstring of T produced by a δ-approximation maximum overlap algorithm. Then:*

$$apx(T) \leq opt(T) + (1 - \delta) maxov(T) \tag{2.16}$$

Proof. Recall the fundamental relationship that $opt(T) = \sum_{t_i \in T} |t_i| - maxov(T)$. Since the overlap achieved by the δ-overlap approximation algorithm is by definition at least $\delta \cdot maxov(T)$, the length of the resulting string is bounded by:

$$apx(T) \leq \sum_{t_i \in T} |t_i| - \delta \cdot maxov(T) = opt(T) + (1 - \delta) maxov(T) \tag{2.17}$$

□

To utilize this lemma, we need a strict upper bound on the possible maximum overlap $maxov(T)$. In older algorithms, the standard bound was $maxov(T) \leq 2CYC(G_S)$. However, due to the specific choice of cycle representatives t_C in Step 3, this bound can be significantly tightened.

Lemma 9. *The maximum overlap of the set T is bounded by:*

$$maxov(T) \leq \frac{3}{2} CYC(G_S) \tag{2.18}$$

Proof. Consider the shortest superstring for T, and assume that it contains the strings $t_1, t_2, \ldots$ in that specific order. Recall that the strings in T are mutually inequivalent due to the minimality of the cycle cover $\mathcal{C}_S$. Therefore, by applying the Overlap-Rotation Lemma 6, the overlap between any two adjacent strings in the superstring satisfies:

$$ov(t_i, t_{i+1}) \leq period(t_i) + \frac{1}{2} period(t_{i+1}) \tag{2.19}$$

Summing these overlaps over all adjacent strings $t_i \in T$, and recalling that the period of a representative string equals the weight of its generating cycle, we obtain:

$$maxov(T) = \sum_{i=1}^{|T|-1} ov(t_i, t_{i+1}) \leq \frac{3}{2} \sum_{t_i \in T} period(t_i) = \frac{3}{2} CYC(G_S) \tag{2.20}$$

□

By combining these bounds and utilizing a δ-approximation algorithm for directed Max-TSP-Path in the final step, the overall approximation ratio is established. Table 2.1 summarizes the history of such algorithms and the resulting shortest superstring ratio obtained by substituting δ into Lemma 8 and Lemma 9.

Theorem 8. *By employing the $\frac{38}{63}$-overlap approximation algorithm proposed by Kosaraju in Step 4, the algorithm achieves a $2\frac{25}{42} \approx 2.596$ approximation for the shortest superstring problem.*

Proof. By substituting the bounds from Lemma 3, Lemma 8, and Lemma 9 into the overlap approximation formula with $\delta = \frac{38}{63}$, the length of the produced superstring is bounded by:

$$apx(T) \leq opt(T) + \left(1 - \frac{38}{63}\right) maxov(T) \tag{2.21}$$

Table 2.1: History of approximation-ratio improvements for directed Max-TSP-Path, and the resulting Shortest Superstring ratio $2+\frac{3}{2}(1-\delta)$ obtained when the algorithm is plugged into Step 4 of this section.

Year	Authors	Max-TSP-Path Ratio δ	Resulting SCS Ratio
1994	**Kosaraju, Park, Stein [KPS94]**	$\mathbf{\frac{38}{63} \approx 0.603}$	$\mathbf{2\frac{25}{42} \approx 2.596}$
2004	Bläser [Blä04]	$\frac{8}{13} \approx 0.615$	$2\frac{15}{26} \approx 2.577$
2005	Kaplan et al. [Kap+05]	$\frac{2}{3} \approx 0.667$	$2\frac{1}{2} = 2.5$
2012	Paluch, Elbassioni, van Zuylen [PEZ12]	$\frac{2}{3} \approx 0.667$	$2\frac{1}{2} = 2.5$
2020	Paluch [Pal20]	$\frac{7}{10} = 0.7$	$2\frac{9}{20} = 2.45$

$$apx(T) \leq 2opt(S) + \left(\frac{25}{63}\right)\frac{3}{2}CYC(G_S) \tag{2.22}$$

Since $CYC(G_S) \leq opt(S)$, this simplifies to:

$$apx(T) \leq 2opt(S) + \frac{25}{42}opt(S) = 2\frac{25}{42}opt(S) \tag{2.23}$$

This mathematically guarantees the 2.596 approximation ratio. □

2.5.3 Computational Complexity

Let $n = |S|$ and let $L = \sum_{s \in S} |s|$ denote the total input length. Steps 1 and 2 build the distance graph G_S and compute its minimum weight cycle cover, the latter using the Hungarian algorithm in $O(n^3)$ time [Kuh55; EK72; Tom71]. As in the $2\frac{2}{3}$-approximation algorithm, Step 3 computes the representatives t_C in $O(L)$ time overall, by Lemma 7. Step 4 runs a δ-approximation algorithm for directed Max-TSP-Path on T, where $|T| \leq n$. Writing $T_\delta(n)$ for the running time of this subroutine, the algorithm overall runs in $O(n^3 + T_\delta(n) + L)$ time.

When Kosaraju's $\frac{38}{63}$-approximation algorithm is used for this step (Theorem 12), each of its three versions relies on the Hungarian algorithm and Edmonds' blossom algorithm, both running in $O(n^3)$ time, so $T_\delta(n) = O(n^3)$. Substituting this back, the algorithm as a whole runs in $O(n^3 + L)$ time.

Chapter 3

Kosaraju-Park-Stein (1994) Algorithms

3.1 Approximating the Longest Tour in a Directed Graph

To construct the overlap approximation subroutine used in $2\frac{25}{42}$-Approximation algorithm, we analyze the $\frac{38}{63}$-approximation algorithm for the directed Max TSP. For the simplicity in the theoretical analysis, we assume throughout this section that the number of vertices in the input graph is even. If the number of vertices is odd, we can add a vertex that is connected to all other vertices with 0-weight edges.

The algorithm heavily relies on two standard polynomial-time graph subroutines:

1. The Hungarian algorithm [Kuh55; EK72; Tom71] for solving the maximum-weight cycle cover problem via its reduction to the assignment problem, running in $O(n^3)$ time.

2. Edmonds' blossom algorithm [Edm65b; Edm65a] for finding a maximum-weight (not necessarily perfect) matching in an undirected graph, also running in $O(n^3)$ time.

3.1.1 The Generic Directed Max TSP Algorithm

The approximation algorithm consists of three distinct versions, all of which share the following framework:

1. **Cycle Cover:** Compute a maximum-weight cycle cover $\mathcal{C}$ for the graph G.

2. **Path Extraction:** Use the cycle cover $\mathcal{C}$ to obtain a set P of vertex-disjoint paths.

3. **Tour Construction:** Construct a final tour by patching together the paths from P.

The three versions of the algorithm differ strictly in their implementation of Step 2. The first version follows the simple approach of deleting the smallest edge in each cycle. The second and third approaches are significantly more complicated, utilizing non-cycle edges in an attempt to extract a set of paths with a greater total weight.

3.1.2 Preliminaries

Before comparing these three approaches, we must establish the following definitions regarding the cycle cover $\mathcal{C}$:

- A **2-cycle** is any cycle passing through exactly 2 vertices.
- A 3^+**-cycle** is any cycle passing through 3 or more vertices.
- W denotes the total weight of all edges in $\mathcal{C}$.
- bW denotes the total weight of the edges that are the heavier (i.e., larger-weight) edge in some 2-cycle of $\mathcal{C}$.
- cW denotes the total weight of the edges that are the lighter (i.e., smaller-weight) edge in some 2-cycle of $\mathcal{C}$.

By expressing the approximation bounds of the three versions as linear functions of the term $(b - 2c)$, we can prove that the best of the three always guarantees at least a $\frac{38}{63}$-approximation.

3.1.3 Crucial Observation

A crucial observation simplifies the mathematical analysis of this framework. To bound the weight of the final produced tour, it is entirely sufficient to obtain a lower bound on $w(P)$, which is the total weight of the paths extracted in Step 2. Because the graph strictly contains non-negative edge weights, no matter how the vertex-disjoint paths of P are patched together in Step 3, the weight of the resulting tour will always be at least $w(P)$. As the result, the explicit operations of Step 3 can be safely ignored in the remainder of the analysis.

3.2 Version 1: Simple Edge Deletion

The first version of the generic algorithm relies on a straightforward approach to break the cycles. To obtain the set of vertex-disjoint paths P from the cycle cover $\mathcal{C}$, the algorithm simply deletes the minimum-weight edge from each cycle.

In the absolute worst-case scenario—where the cycle cover $\mathcal{C}$ consists entirely of 2-cycles and both edges in every 2-cycle have identical weights (i.e., $b = c$)—the weight of the resulting paths $w(P)$ can be as small as $\frac{1}{2}W$. However, in most instances, the presence of larger cycles and weight imbalances ensures a significantly better result.

To analyze this formally, let pW denote the total weight of the set consisting of the lightest edge from each 3^+-cycle in $\mathcal{C}$. Because we remove exactly the lighter edge from each 2-cycle (totaling cW) and the lightest edge from each 3^+-cycle (totaling pW), the total weight of the remaining paths is precisely:

$$w(P) = (1 - c - p)W \tag{3.1}$$

By definition, the lightest edge in any 3^+-cycle can contain at most one-third of the total weight of that cycle. Since the total weight of all 3^+-cycles in $\mathcal{C}$ is exactly $(1-b-c)W$, we can bound p by the inequality:

$$p \leq \frac{1}{3}(1-b-c) \tag{3.2}$$

By substituting the upper bound from Inequality 3.2 into the equation for $w(P)$, we can establish the guaranteed approximation ratio for this version.

Theorem 9. *Version 1 of the generic algorithm returns a* $\left(\frac{2}{3}+\frac{1}{3}(b-2c)\right)$*-approximation to the directed Max TSP.*

Proof. Substituting $p \leq \frac{1}{3}(1-b-c)$ into the weight function gives:

$$w(P) \geq \left(1-c-\frac{1}{3}(1-b-c)\right)W = \left(\frac{2}{3}+\frac{1}{3}(b-2c)\right)W \tag{3.3}$$

Since the weight of the optimal tour $w(T_{opt})$ is bounded from above by the weight of the optimal cycle cover W, the resulting set of paths P guarantees the stated approximation ratio. □

3.2.1 Computational Complexity

Let n denote the number of vertices of G. Computing the maximum-weight cycle cover $\mathcal{C}$ via the Hungarian algorithm takes $O(n^3)$ time [Kuh55; EK72; Tom71]. Since the cycles of $\mathcal{C}$ partition the vertex set, deleting the minimum-weight edge from each cycle – and thereby obtaining the set of paths P – takes only $O(n)$ time overall, as does patching the paths of P into a single tour. The cycle cover computation therefore dominates, and Version 1 runs in $O(n^3)$ time.

3.3 Version 2: Path Coloring

To extract a heavier set of paths from the cycle cover, the second iteration of the generic algorithm utilizes non-cycle edges. To establish the core mechanics, we first present a simplified variant before introducing the fully optimized version that achieves the stronger approximation bound.

3.3.1 The Simple Variant of Version 2

The basic variant operates through the following sequence of steps:

Step 2a. Form an auxiliary undirected graph G' derived from the base graph G and its cycle cover $\mathcal{C}$ in the following manner:

- Substitute each 2-cycle $\{(u,v),(v,u)\}$ of $\mathcal{C}$ with an undirected edge $\{u,v\}$ carrying a weight of 0.

- For all remaining pairs of vertices u and v, introduce an undirected edge $\{u, v\}$ whose weight is equal to $\max\{w(u, v), w(v, u)\}$.

Step 2b. Compute a maximum-weight matching M within this auxiliary graph G'.

Step 2c. Revert the edges of M to their directed forms, and augment this set by selecting a single edge from every 3^+-cycle (ensuring no cycles are created in the corresponding undirected graph).[1]

Step 2d. Contract the 2-cycles into single vertices.

Step 2e. Two-color edges of the remaining graph to divide it into two sets of directed paths. Form the final path collection P by taking the edges from the heavier color group and supplementing them with one compatible edge extracted from each 2-cycle.

Analysis of the Simple Variant Let T_{opt} represent the optimal maximum-weight tour, with $w(T_{opt})$ denoting its total weight. When evaluating the weight of T_{opt} within the context of G', we note that 2-cycles are assigned a weight of 0, while all other edges maintain or increase their weights. As a result, the weight of T_{opt} in G' is bounded below by $w(T_{opt}) - bW$, given that each 2-cycle can contribute at most one edge to T_{opt}.

Because T_{opt} is a single cycle on an even number of vertices, its edges can be alternately split into two matchings averaging at least $\frac{1}{2}(w(T_{opt}) - bW)$. Therefore, the maximum-weight matching M obtained in Step 2b is at least this average weight.

Adding one edge from each 3^+-cycle in Step 2c contributes an additional weight of pW. The total weight of this constructed set of edges is therefore bounded below by:

$$\alpha = \frac{w(T_{opt}) - bW}{2} + pW \tag{3.4}$$

Contracting the 2-cycles in Step 2d does not alter this total weight, as they were initially zero-weighted in G'. We are then left with a collection of paths and cycles. These edges can be two-colored (partitioned into two color classes) such that each color class forms a collection of vertex-disjoint directed paths. Consequently, the heavier of these two classes must contain a total weight of at least $\frac{\alpha}{2}$.

Upon uncontracting the 2-cycles, it is mathematically guaranteed that we can always select one edge per 2-cycle to add to our chosen color class while maintaining a valid collection of disjoint paths. Assume, for the contradiction, that this was impossible. This would imply that a 2-cycle has two incident edges of the same color, both directed either towards or away from the cycle. However, in the contracted graph, these two edges would have been directed either toward or away from the contracted vertex—meaning they could not have validly received the same color in a path coloring, establishing a contradiction.

[1] Proof that it can be done is omitted, because in the real variant we use the stronger Lemma 10.

This final addition gives a set of directed paths with a minimum weight of:

$$\frac{\alpha}{2} + cW = \frac{w(T_{opt}) - bW}{4} + \frac{pW}{2} + cW \tag{3.5}$$

Substituting the bounds $p \leq \frac{1}{3}(1-b-c)$ and $W \geq w(T_{opt})$ into this equation results in a bound of $\frac{5}{4}(c+p)w(T_{opt})$.

By combining this bound with the $(1-c-p)w(T_{opt})$ bound from Version 1 (Theorem 9) – taking whichever of the two constructions yields the heavier set of paths – it guarantees a weight of at least:

$$\max\left\{(1-c-p),\ \frac{5}{4}(c+p)\right\} w(T_{opt}). \tag{3.6}$$

Writing $x = c + p$, the first term is decreasing in x and the second is increasing in x, so their maximum is minimized exactly where the two coincide:

$$1 - x = \frac{5}{4}x \iff x = \frac{4}{9}, \tag{3.7}$$

at which point both bounds equal $1 - x = \frac{5}{9}$. Hence, in the worst case over all valid b, c, p, this basic variant is guaranteed to achieve a $\frac{5}{9}$-approximation for the Max TSP.

3.3.2 Version 2: The Real Variant

To secure a stronger approximation bound, the simplified algorithm requires two major improvements. First, rather than assigning zero weight to the 2-cycle edges (which only guarantees the inclusion of the lighter edge), we allow a 2-cycle edge to be placed in the matching if the heavier edge significantly outweighs the lighter edge. To achieve this, we assign a weight of $2(b_i - c_i)$ to the undirected edge representing a 2-cycle C_i, where b_i and c_i are the weights of the heavy and light edges, respectively. Second, a specialized path-coloring lemma is utilized, permitting the inclusion of every edge in a 3^+-cycle except the lightest.

Algorithm: Version 2, Real Variant

Step 2a. Build the undirected graph G' from G and $\mathcal{C}$ with the following adjustments:

- Reclassify any 2-cycle where $b_i > 2c_i$ as a 3^+-cycle.
- Replace each 2-cycle $\{(u,v),(v,u)\}$ of $\mathcal{C}$ with an undirected edge $\{u,v\}$ carrying a weight of $2(b_i - c_i)$.
- For all other pairs of vertices u and v, create an undirected edge $\{u,v\}$ with weight $\max\{w(u,v), w(v,u)\}$.

Step 2b. Extract a maximum-weight matching M from G'.

Step 2c. To the directed versions of M's edges, attach every edge besides lightest from the 3^+-cycles.

Step 2d. Contract the 2-cycles found in $\mathcal{C}$.

Step 2e. Two-color edges of the remaining graph using Lemmas 10 and 11 to generate two sets of directed paths. The final set P comprises the heavier color group combined with one edge from each 2-cycle.

We start by stating the primary coloring lemma, referring to the resulting coloring as a *path coloring*. Proof of this was omitted in the original paper, but since it is a complex result, we devote Chapter 4 to its elaboration.

Lemma 10 (Path-Coloring Lemma [KPS94])**.** *Let G be a directed graph satisfying two conditions: (1) each vertex has an indegree of at most 2, an outdegree of at most 2, and a total degree of at most 3, and (2) the graph contains no 2-cycles. Under these conditions, the edges of G can be 2-colored such that each color class forms a collection of directed paths.*

Algorithm Analysis We introduce four weight quantities describing how the edges of T_{opt} interact with the 2-cycles of $\mathcal{C}$:

- b_1W – the total weight of the edges in T_{opt} that are the heavier edge of a 2-cycle in $\mathcal{C}$
- c_1W – the total weight of the corresponding (lighter) sibling edges of those same 2-cycles
- c_2W – the total weight of the edges in T_{opt} that are the lighter edge of a 2-cycle in $\mathcal{C}$
- b_2W – the total weight of the corresponding (heavier) sibling edges of those same 2-cycles

The weight of the optimal tour edges not belonging to a 2-cycle in $\mathcal{C}$ is $w(T_{opt}) - b_1W - c_2W$. By adding back the modified weight of the 2-cycle edges, the total weight of the optimal tour in G' is bounded below by:

$$w(T_{opt}) - b_1W - c_2W + 2(b_1 - c_1)W + 2(b_2 - c_2)W$$

Given that each 2-cycle C_i satisfies $c_i \leq b_i \leq 2c_i$, it follows that:

$$(b_1 - 2c_1 + 2b_2 - 3c_2) \geq (b_1 + b_2) - 2(c_1 + c_2) \geq b - 2c$$

This implies the optimal tour's weight in G' is at least $w(T_{opt}) + (b - 2c)W$. As a result, the matching extracted in Step 2b has a weight of at least $\frac{1}{2}(w(T_{opt}) + (b - 2c)W)$.

In Step 2c, including the 3^+-cycles adds a weight of $(1 - p - b - c)W$. Therefore, prior to Step 2d, the graph's total weight is at least:

$$\frac{w(T_{opt}) + (b - 2c)W}{2} + (1 - p - b - c)W \tag{3.8}$$

To evaluate the remaining weight after Step 2d, let A denote the set of 2-cycles with corresponding edges in the matching. Let $\hat{b}W$ and $\hat{c}W$ be the sum of the weights of the

heavier and lighter edges in A, respectively. Simplifying equation (3.8) and subtracting the contracted weight $2(\hat{b}-\hat{c})W$, we are left with an edge collection at the end of Step 2d weighing at least:

$$\frac{w(T_{opt})}{2} + \left(1 - 2c - \frac{b}{2} - p - 2(\hat{b}-\hat{c})\right) W \tag{3.9}$$

Handling Induced 2-Cycles The edges at the start of Step 2e consist of paths originating from the 3^+-cycles combined with matching edges. While this graph violates the conditions of Lemma 10, the following lemma proves it can still be path-colored with two colors.

Lemma 11 ([KPS94])**.** *The set of edges E remaining after Step 2d can be successfully path-colored using two colors.*

Proof. By design, every vertex has at most one incoming cycle edge, one outgoing cycle edge, and one matching edge, satisfying the degree constraints of Lemma 10. Moreover, contracting the 2-cycles of $\mathcal{C}$ merges two vertices (each of degree at most 3, sharing 2 edges) into a single vertex of degree at most 2. This satisfies all conditions of Lemma 10 except for the potential presence of 2-cycles. These remaining 2-cycles, formed solely by one cycle edge and one matching edge, are referred to as *induced 2-cycles*.

Consider such an induced 2-cycle $\{(u,v),(v,u)\}$. Due to the degree-three limit and the presence of a matching edge between u and v, there is at most one additional edge connected to v (labeled (v,x)) and at most one additional edge connected to u (labeled (u,y)), neither being a matching edge. To resolve the 2-cycle, the path (x,v,u,y) is contracted into a single edge (x,y). The resulting graph retains the same properties but contains one fewer 2-cycle. This process is repeated until all 2-cycles are eliminated. Once the contracted graph is colored, coloring the original graph is straightforward: if (x,y) is colored red, then (x,v), (v,u), and (u,y) are colored red, while (u,v) is assigned blue. Because (u,v) shares no vertices with another blue edge, this assignment is perfectly safe. □

Final Bound Given that the edge weight remaining after Step 2d's contractions is bounded by equation (3.9), the preceding lemma allows us to secure a collection of directed paths with a weight of at least:

$$\frac{1}{2}\left(\frac{w(T_{opt})}{2} + \left(1 - 2c - \frac{b}{2} - p - 2(\hat{b}-\hat{c})\right) W\right) \tag{3.10}$$

The final step—adding 2-cycle edges at the conclusion of Step 2e—increases the value in (3.10) by a minimum of $(\hat{b}+(c-\hat{c}))W$. This is because, for each 2-cycle in $\mathcal{C}$, we include its heavier edge if the corresponding undirected edge was in M, or an edge weighing at least as much as the lighter edge if it was not. Since $\mathcal{C}$ is a cycle cover, the vertices of a 2-cycle belong to no other cycle, so if its collapsed edge was chosen in M, both endpoints are matched only to each other, the contracted vertex is then completely isolated in the graph colored in Step 2e, and either directed edge can be added back without conflict, so we pick the heavier one. Otherwise, at least one endpoint may carry an external matching

edge, which fixes which direction is safe to add – we can then only guarantee the weight of the lighter edge. Finally, this gives a path collection with a weight of at least:

$$\frac{1}{2}\left(\frac{w(T_{opt})}{2} + \left(1 - 2c - \frac{b}{2} - p - 2(\hat{b} - \hat{c})\right) W\right) + (\hat{b} + (c - \hat{c}))W$$
$$\geq \left(\frac{3}{4} - \frac{b}{4} - \frac{p}{2}\right) w(T_{opt}) \geq \left(\frac{7}{12} - \frac{1}{12}(b - 2c)\right) w(T_{opt}).$$

This final inequality is derived by applying the bound $p \leq \frac{1}{3}(1 - b - c)$, which remains valid even after reclassifying 2-cycles with $b_i > 2c_i$ as 3^+-cycles. This leads to the final lemma.

Theorem 10. *Version 2 of the generic algorithm guarantees a* $\left(\frac{7}{12} - \frac{1}{12}(b - 2c)\right)$*-approximation for the Max TSP.*

Although this bound is formulated using the values of b and c obtained after reclassifying specific 2-cycles, this reclassification can only decrease the value of $b - 2c$. Therefore, the bound established in Theorem 10 is strictly valid for the original parameters of b and c as well.

Computational Complexity Let n denote the number of vertices of G. Building the auxiliary graph G' in Step 2a takes $O(n^2)$ time. Extracting the maximum-weight matching M in Step 2b, using Edmonds' blossom algorithm, takes $O(n^3)$ time [Edm65b; Edm65a]. Steps 2c and 2d are linear in n. The path coloring of Step 2e executes deterministically in $O(n^3)$ time, as established in Chapter 4. The matching computation and path coloring therefore dominate, and Version 2 runs in $O(n^3)$ time overall.

3.4 Version 3: Cycle Contraction and Compatible Weights

The third variant of the generic algorithm shifts its strategy to aggressively capture edge weight that was entirely missed by the initial cycle cover $\mathcal{C}$, while simultaneously preserving as much weight from the cycles as possible. Again to introduce the mechanics and mathematical framework of this approach, we first perform an analysis of a simplified variant.

The core intuition relies on constructing an auxiliary directed multigraph and modifying edge weights to reflect the "compatibility" between external edges and the edges within the cycle cover.

3.4.1 The Compatible Weight Metric

For any cycle $C \in \mathcal{C}$ and an edge e with exactly one endpoint incident to C, we define $cw(e, C)$, the *compatible weight* of C with respect to e, as follows:

$$cw(e, C) = w(C) - w(f_{e,C})$$

where $f_{e,C}$ is the unique edge in C that is structurally incompatible with e (i.e., it shares either a head or a tail with e).

When we form a new auxiliary directed multigraph $G' = (V', E')$ by contracting the cycles of $\mathcal{C}$ into single vertices, we define a new weight metric $w'(e)$ for every edge $e \in E'$. This new weight aggregates the original weight of the edge and the compatible weights of the cycles corresponding to its endpoints:

$$w'(e) = w(e) + cw(e, C_{head(e)}) + cw(e, C_{tail(e)})$$

where $C_{head(e)}$ and $C_{tail(e)}$ represent the specific cycles in G that contain the head and tail of edge e, respectively. By definition, a matching M found in G' corresponds naturally to a set of disjoint paths P in the original graph G, such that the actual weight of the paths equals the modified weight of the matching: $w(P) = w'(M)$.

3.4.2 Algorithm: Version 3, Simple Variant

The simplified variant proceeds via the following steps:

Step 2a. Construct an auxiliary directed multigraph $G' = (V', E')$ by contracting each cycle $C \in \mathcal{C}$ to a single vertex. Assign the edge weights w' for G' as defined above.

Step 2b. Find a maximum-weight matching M in G'. Let P be the set of paths in G corresponding to the edges of M.

3.4.3 Mathematical Analysis of the Simple Variant

To analyze the weight of the extracted paths P, we define γ as the maximum number of vertices present in any single cycle within $\mathcal{C}$. Let T_{opt} denote the optimal Max TSP tour, and let T_{ext} denote the subset of edges from T_{opt} that have endpoints in distinct cycles of $\mathcal{C}$ (i.e., external edges).

Assuming No Internal Tour Edges We begin our analysis by assuming that $T_{opt} = T_{ext}$, meaning all edges of the optimal tour travel between distinct cycles in the cycle cover. We can bound the total modified weight of the optimal tour in G' by summing the modified weights of all its edges:

$$w'(T_{opt}) = \sum_{e \in T_{opt}} w'(e) = \sum_{e \in T_{opt}} \big(w(e) + cw(e, C_{head(e)}) + cw(e, C_{tail(e)})\big)$$

By substituting the definition of compatible weight and regrouping the terms by cycle, we obtain:

$$w'(T_{opt}) = w(T_{opt}) + \sum_{C \in \mathcal{C}} \sum_{\substack{e \in T_{opt} \text{ s.t.} \\ head(e) \in C \text{ or } tail(e) \in C}} (w(C) - w(f_{e,C}))$$

Because T_{opt} is a valid tour, each cycle C will have exactly $2|C|$ incident tour edges ($|C|$ entering and $|C|$ leaving). Furthermore, each internal edge of C will be structurally incompatible with exactly two of these tour edges. Therefore, the summation simplifies to:

$$w'(T_{opt}) = w(T_{opt}) + \sum_{C \in \mathcal{C}} (2|C| - 2)w(C)$$

Applying Vizing's Theorem and Unmatched Vertices In the auxiliary graph G', no vertex can have more than 2γ edges from T_{opt} incident to it. According to Vizing's Theorem [Viz64], this implies that the edges of T_{opt} in G' can be properly colored using $(2\gamma+1)$ colors. Thus, the edges of T_{opt} can be partitioned into $(2\gamma+1)$ distinct matchings, giving an average weight of at least $\frac{1}{2\gamma+1}w'(T_{opt})$.

However, we can make a significantly stronger mathematical claim by considering unmatched vertices. In any specific color class, a vertex v_C (representing cycle C) might not have any incident edges. If v_C is unmatched, it means no vertex within cycle C has an incident edge in the corresponding path set P. In this scenario, we can augment P by independently adding all but the single lightest edge of C. If we let $w'(C)$ denote the weight of all edges in C except the lightest, we can guarantee that $w'(C) \geq \frac{|C|-1}{|C|}w(C)$.

A vertex v_C has exactly $2|C|$ incident edges in T_{opt}, meaning it is matched in exactly $2|C|$ of the color classes. Consequently, it must remain unmatched in exactly $(2\gamma+1-2|C|)$ of the $(2\gamma+1)$ matchings. Summing these guaranteed additions over all $2\gamma+1$ matchings, we establish that the maximum-weight matching M must satisfy:

$$w'(M) \geq \frac{1}{2\gamma+1}\left(w(T_{opt}) + \sum_{C \in \mathcal{C}} (2|C| - 2)w(C) + \sum_{C \in \mathcal{C}} (2\gamma + 1 - 2|C|)\frac{|C|-1}{|C|}w(C)\right) \tag{3.11}$$

$$= \frac{1}{2\gamma+1}\left(w(T_{opt}) + \sum_{C \in \mathcal{C}} (2\gamma + 1)\frac{|C|-1}{|C|}w(C)\right) \tag{3.12}$$

For the purpose of analysis we refer to:

- $\sum_{C \in \mathcal{C}} (2|C| - 2)w(C)$ as the cycle's *colored contribution*,
- $\sum_{C \in \mathcal{C}} (2\gamma + 1 - 2|C|)\frac{|C|-1}{|C|}w(C)$ as its *uncolored contribution*.

Handling Internal Tour Edges We now remove the simplifying assumption and account for cases where $T_{opt} \neq T_{ext}$. Because only the edges of T_{ext} appear in G', we must replace $w(T_{opt})$ with $w(T_{ext})$ in our formula.

When an edge of T_{opt} is internal to a cycle C, the vertex representing C in G' has two fewer incident edges from the optimal tour. This decreases the cycle's colored contribution by $2w(C) - w(e_1) - w(e_2)$ (where e_1 and e_2 are edges of C). However, because the vertex's degree in G' has dropped by two, it is now uncolored two additional times. This increases

its uncolored contribution by $2w'(C) = 2w(C) - 2w(e_{lightest})$, where $e_{lightest}$ is the lightest edge of the cycle C. The net effect on the bound is the increase in uncolored contribution minus the decrease in colored contribution:

$$(2w(C) - 2w(e_{lightest})) - (2w(C) - w(e_1) - w(e_2)) = (w(e_1) + w(e_2)) - 2w(e_{lightest}). \tag{3.13}$$

Because the weights of any edges $e_1, e_2 \in C$ must be greater than or equal to $w(e_{lightest})$, this net effect is always non-negative – the gain in uncolored contribution never shrinks by more than the loss in colored contribution. Therefore, the inequality remains valid when adjusted for external edges:

$$w'(M) \geq \frac{1}{2\gamma + 1} \left(w(T_{ext}) + \sum_{C \in \mathcal{C}} (2\gamma + 1) \frac{|C| - 1}{|C|} w(C) \right) \tag{3.14}$$

To relate this back to the total optimal tour weight, let $T_{int}(C)$ denote the set of T_{opt} edges completely internal to cycle C. It must hold that $w(T_{int}(C)) \leq w(C)$ otherwise we could patch edges of $T_{int}(C)$ to cycles which would give a cycle cover with strictly greater weight, violating the definition of a maximum-weight cycle cover. We can always do it because our graph is complete and all edges have non negative weights. Thus, $w(T_{ext}) \geq w(T_{opt}) - \sum_{C \in \mathcal{C}} w(C)$. Substituting this lower bound into equation (3.14) gives:

$$w'(M) \geq \frac{1}{2\gamma + 1} \left(w(T_{opt}) + \sum_{C \in \mathcal{C}} \left((2\gamma + 1) \frac{|C| - 1}{|C|} - 1 \right) w(C) \right) \tag{3.15}$$

Limitations of the Simple Variant If we plug the worst-case lower bound of $|C| \geq 2$ into equation (3.15), the result simplifies to show only that $w'(M) \geq \frac{1}{2} w(T_{opt})$. Because this provides no improvement over the bound established in Version 1, a more sophisticated approach is required.

To see this explicitly: since $\frac{|C|-1}{|C|}$ is increasing in $|C|$ and every cycle satisfies $|C| \geq 2$, each coefficient in equation (3.15) is minimized at $|C| = 2$, giving

$$(2\gamma + 1) \frac{|C| - 1}{|C|} - 1 \geq (2\gamma + 1) \cdot \frac{1}{2} - 1 = \gamma - \frac{1}{2}. \tag{3.16}$$

Applying this bound to every term of the sum, and recalling $\sum_{C \in \mathcal{C}} w(C) = W$, gives

$$w'(M) \geq \frac{1}{2\gamma + 1} \left(w(T_{opt}) + \left(\gamma - \frac{1}{2} \right) W \right). \tag{3.17}$$

Substituting the worst case $W = w(T_{opt})$ (the smallest value consistent with $W \geq w(T_{opt})$) yields

$$w'(M) \geq \frac{1}{2\gamma + 1} \left(w(T_{opt}) + \left(\gamma - \frac{1}{2} \right) w(T_{opt}) \right) = \frac{\gamma + \frac{1}{2}}{2\gamma + 1} w(T_{opt}) = \frac{1}{2} w(T_{opt}), \tag{3.18}$$

since $\gamma + \frac{1}{2} = \frac{2\gamma+1}{2}$. Notably, γ cancels out entirely, so this weak bound holds regardless of the largest cycle size.

3.4.4 Version 3: The Real Variant

To overcome the mathematical limitations of the simple variant, the "Real Variant" introduces a critical modification: bounding the maximum size of any cycle. Additionally, it refines the compatible weight metric by utilizing Hamiltonian paths.

It is worth noting that simply plugging a bound on γ into the previous formula would not help: as shown above, the weak bound $w'(M) \geq \frac{1}{2}w(T_{opt})$ holds regardless of γ, since γ cancels out entirely. The real issue is that Vizing's Theorem forces $(2\gamma + 1)$ colors for the *entire* auxiliary graph G', based on whichever single cycle happens to be largest – so even a cycle of moderate size gets diluted by dividing its contribution by this same, potentially huge, $2\gamma + 1$. By splitting every cycle larger than a constant x into pieces of size at most x, every piece becomes small enough to be analyzed with the *same* Vizing-based colored/uncolored contribution formula, but now with x (a constant we control) in place of the unbounded γ – this is exactly what Lemma 13 does. 2-cycles are treated separately still: they bypass the coloring argument entirely, via the dedicated case analysis of Lemma 14, since their two edges admit a direct combinatorial argument that a generic coloring bound cannot capture as tightly.

Specifically, the compatible weight $cw(e, C)$ is redefined as $w(H_{e,C})$, where $H_{e,C}$ is a maximum-weight Hamiltonian path spanning the vertices of C that remains structurally compatible with the external edge e. Similarly, the uncolored weight of a cycle is updated to $w(H_C)$, representing an unrestricted maximum-weight Hamiltonian path on the vertices of C. Since the cycle size will be bounded by a constant, computing these Hamiltonian paths requires only constant time per cycle.

Algorithm: Version 3, Real Variant

Step 2a. Define a constant x (to be specified later). For every cycle $C \in \mathcal{C}$ where $|C| > x$, divide it into $\lceil |C|/x \rceil$ distinct segments. This division must ensure that all resulting pieces, with the possible exception of at most one, contain exactly x vertices. Let $\mathcal{C}'$ denote this updated collection of cycles and paths.

Step 2b. Construct an auxiliary directed multigraph $G' = (V', E')$ by contracting each element within $\mathcal{C}'$ into a single corresponding vertex. Assign edge weights w' across G' using the updated Hamiltonian-based metric described above.

Step 2c. Compute a maximum-weight matching M within G'. Define P as the collection of vertex-disjoint paths in the original graph G that correspond to the edges in M.

Analysis of Cycle Splitting Breaking cycles into smaller segments definitly causes some loss of weight. However, this loss can be strictly bounded, as established by the following lemma.

Lemma 12. *During Step 2a, any given cycle C can be partitioned into a set of pieces P_i such that the total weight retained satisfies:*

$$\sum_{P_i \in C} w(P_i) \geq \frac{|C| - \lceil |C|/x \rceil}{|C|} w(C) \geq \frac{x-1}{x+1} w(C)$$

Proof Sketch. Write $l = |C|$ and $k = \lceil l/x \rceil \geq 2$. Recall that Step 2a cuts C into pieces of (at most) x consecutive vertices, rather than removing an arbitrary set of k edges we must therefore show that some such cut retains enough weight. For each offset $s \in \{0, 1, \ldots, x-1\}$, cutting every x-th edge of C starting from s yields exactly this kind of partition. Since each edge of C is cut for exactly one offset s, the total weight cut summed over all x offsets equals $w(C)$ exactly, so the average weight cut per offset is $w(C)/x$. As $k \geq l/x$, this average is at most $(k/l)w(C)$, so some offset cuts a total weight of at most $(k/l)w(C)$ using this offset retains at least $w(C) - (k/l)w(C) = \frac{l-k}{l} w(C)$, establishing the first inequality. The worst-case scenario resulting in the highest fraction of lost weight occurs when exactly 2 edges must be removed from a cycle consisting of $x+1$ edges.

To see this, multiplying the desired inequality $\frac{l-k}{l} \geq \frac{x-1}{x+1}$ by $l(x+1) > 0$ gives $(l-k)(x+1) \geq l(x-1)$ expanding both sides yields $lx + l - kx - k \geq lx - l$, and subtracting lx then adding l to both sides simplifies this to

$$2l - kx - k \geq 0 \iff 2l \geq k(x+1).$$

By definition of the ceiling, $l \geq x(k-1)+1$, so it suffices to check this bound at its smallest value:

$$2\big(x(k-1)+1\big) \geq k(x+1) \iff kx - k \geq 2x - 2 \iff k(x-1) \geq 2(x-1) \iff k \geq 2,$$

which always holds once splitting occurs. Equality holds precisely when $k = 2$, i.e., $l = x+1$, confirming the worst case and establishing the second inequality of the lemma. □

For analytical purposes, there is a fundamental difference between an original unsplit cycle and a newly created piece P_i resulting from a split. Because P_i is structurally a path rather than a closed cycle, its entire weight can be recovered if it remains unmatched in the auxiliary graph. Consequently, for any piece P_i, the following inequality holds:

$$w'(P_i) \geq w(P_i) \tag{3.19}$$

Net Contribution of Cycles To accurately evaluate the matching, we calculate the *net contribution* for each element. Defined as the sum of its colored and uncolored contributions, minus its internal weight. The following lemmas establish lower bounds for different categories of cycles.

Lemma 13. *For any original cycle $C \in \mathcal{C}$ where $|C| > x$, the aggregate net contribution*

of the pieces P_i that constitute C is bounded from below by:

$$(2x-2)\frac{x-1}{x+1}w(C)$$

Proof. Given that the redefined edge weights are at least as large as those in the simple variant, the combined colored and uncolored contributions 3.11 of any piece P_i sum to a minimum of:

$$(2|P_i|-2)w(P_i)+(2x+1-2|P_i|)w'(P_i)$$

Furthermore, similar to 3.14 we need to consider also edges internal to P_i. The weight of a maximum-weight Hamiltonian path $w(H_{P_i})$ serves as an upper bound for the weight of the T_{opt} edges internal to P_i. Because $w'(P_i)$ shares this bound, the total net contribution of all pieces forming C is at least:

$$\sum_{P_i \in C}\big((2|P_i|-2)w(P_i)+(2x+1-2|P_i|)w'(P_i)-w'(P_i)\big)$$

By applying inequality (3.19) and simplifying the algebraic terms, the lemma is proven. □

A separate analysis is required for 2-cycles.

Lemma 14 ([KPS94]). *Consider a 2-cycle C_i consisting of a heavier edge with weight b_i and a lighter edge with weight c_i. This cycle provides a net contribution of at least $(2x-1)b_i+c_i$.*

The proof evaluating the three potential traversal patterns of the optimal tour through C_i is omitted.

Aggregating the Final Bound We can now synthesize these components to construct a lower bound for the total weight of the matching M. We utilize Lemma 13 to evaluate cycles where $|C|>x$, Lemma 14 for 2-cycles, and the original fractional bound 3.12 for intermediate cycles (sizes 3 through x), leveraging the fact that $|C|\geq 3$.

Summing these respective net contributions gives:

$$\begin{aligned} w'(M) \geq \frac{1}{2x+1}\Bigg(& w(T_{opt}) + \sum_{|C_i|=2}((2x-1)b_i+c_i) \\ & + \sum_{3\leq|C|\leq x}\left((2x+1)\frac{2}{3}-1\right)w(C) \\ & + \sum_{|C|>x}(2x-2)\frac{x-1}{x+1}w(C)\Bigg) \end{aligned}$$

Letting bW and cW represent the aggregate weights of the heavy and light edges of all

2-cycles respectively, we can simplify this expression into a global bound:

$$w'(M) \geq \frac{1}{2x+1}\left(w(T_{opt}) + (2x-1)bW + cW + \min\left\{\frac{4}{3}x - \frac{1}{3}, (2x-2)\frac{x-1}{x+1}\right\}(1-b-c)W\right)$$

To maximize this approximation ratio, we fix the constant parameter at $x = 7$.

Theorem 11. *By evaluating the aggregated bound at $x = 7$, Version 3 of the generic algorithm guarantees a $\left(\frac{2}{3} + \frac{4}{15}(b-2c)\right)$-approximation for the Max TSP.*

Computational Complexity Let n denote the number of vertices of G. Step 2a splits every cycle in $O(n)$ time. Since x is a fixed constant, computing the compatible weights of Step 2b – which reduce to maximum-weight Hamiltonian paths on pieces of at most x vertices – takes only constant time per piece, so building the auxiliary graph G' takes $O(n^2)$ time overall. Extracting the maximum-weight matching M in Step 2c, using Edmonds' blossom algorithm on G', takes $O(n^3)$ time [Edm65b; Edm65a]. The matching computation therefore dominates, and Version 3 runs in $O(n^3)$ time overall.

3.5 The Combined Approximation Ratio

Theorem 12. *Executing versions 1, 2, and 3 of the generic algorithm and selecting the optimal result guarantees a $\frac{38}{63}$-approximation for the Max TSP problem.*

Proof. Write $y = b - 2c$. The three bounds are linear in y: $f_1(y) = \frac{2}{3} + \frac{1}{3}y$ (Theorem 9), $f_2(y) = \frac{7}{12} - \frac{1}{12}y$ (Theorem 10), $f_3(y) = \frac{2}{3} + \frac{4}{15}y$ (Theorem 11). Taking the best of the three guarantees $\max\{f_1, f_2, f_3\}$, so the worst case is its minimum over y.

Since f_1, f_3 increase in y (with f_1 steeper) and f_2 decreases, this minimum occurs where the decreasing curve meets the shallower increasing one, f_3. Setting $f_2(y) = f_3(y)$:

$$\frac{7}{12} - \frac{1}{12}y = \frac{2}{3} + \frac{4}{15}y \iff y = -\frac{5}{21}, \qquad f_3\left(-\frac{5}{21}\right) = \frac{2}{3} - \frac{4}{63} = \frac{38}{63}. \tag{3.20}$$

At this y the first bound stays below the other two,

$$f_1\left(-\frac{5}{21}\right) = \frac{2}{3} - \frac{5}{63} = \frac{37}{63} < \frac{38}{63}, \tag{3.21}$$

so it does not interfere. Hence $\max\{f_1(y), f_2(y), f_3(y)\} \geq \frac{38}{63}$ for all y. □

Computational Complexity Running versions 1, 2, and 3 and keeping the best result costs the sum of their individual running times, each of which is $O(n^3)$, as established in the complexity analysis of each version above. Since a constant number of $O(n^3)$-time algorithms sum to $O(n^3)$, the generic algorithm as a whole runs in $O(n^3)$ time.

Chapter 4

Path Coloring

For completeness, in this chapter we construct an algorithm proving the Path-Coloring Lemma, which is required as a subroutine in Version 2 of the Kosaraju-Park-Stein algorithm (Chapter 3). Kosaraju, Park, and Stein [KPS94] originally stated this lemma without proof, giving neither an existence argument nor a running time bound. The algorithm and proof presented here follow the approach of Bläser [Blä04], who gave the first rigorous, algorithmic proof of the lemma.

4.1 The Path-Coloring Lemma

Lemma 15 (Path-Coloring Lemma [Blä04])**.** *Let G be a directed graph satisfying the following structural properties:*

1. *Every vertex has an indegree of at most 2.*
2. *Every vertex has an outdegree of at most 2.*
3. *The total degree of any vertex (indegree plus outdegree) is at most 3.*
4. *The graph G contains no 2-cycles.*

Then the edges of G can be colored using exactly two colors such that the edges in each color class form a collection of directed paths.

4.2 Algorithmic Proof of the Path-Coloring Lemma

Let $G = (V, E)$ be the directed graph we intend to color, satisfying the degree and 2-cycle restrictions outlined previously.

4.2.1 Phase 1: Constructing the Initial Coloring

The first phase aims to partition the edges of G into two color classes such that no node acts as a junction for two edges of the same color pointing in the same direction.

We construct an auxiliary undirected graph $A_1 = (E, F_1)$. Every edge $e \in E$ of the original graph G becomes a vertex in A_1 (referred to as an *A-node*). The edge set F_1 (containing *A-edges*) is constructed to capture structural conflicts: an undirected edge $\{e, f\} \in F_1$ exists if and only if the directed edges e and f share either a common head or a common tail in G. Specifically, $\{e, f\} \in F_1$ if there exist vertices $u, v, w \in V$ such that either $e = (u, v)$ and $f = (w, v)$, or $e = (v, u)$ and $f = (v, w)$. This guarantees that colliding edges are forced into separate color classes.

Because the maximum total degree of any vertex in G is bounded by 3, every A-node in A_1 has a degree of at most 2. As a result, the auxiliary graph A_1 decomposes entirely into a collection of isolated vertices, simple paths (A-paths), and simple cycles (A-cycles).

Crucially, every A-cycle in A_1 must contain an even number of A-edges. This property arises because traversing an A-cycle corresponds to alternating between "head-meets" and "tail-meets" in the original graph G. This structural guarantee allows us to trivially 2-color the A-nodes by simply alternating colors along each A-path and A-cycle.

Lemma 16. *The edges of G can be colored with two colors such that each color class forms a collection of node-disjoint paths and cycles. Furthermore, this initial coloring can be computed in polynomial time.*

While Lemma 16 successfully isolates paths and cycles, the path-coloring property requires the complete elimination of all monochromatic cycles. Note that for any A-path or A-cycle in A_1, there are exactly two valid alternating colorings, and we initially choose one arbitrarily. This flexibility is the key observation.

4.2.2 Phase 2: Refining the Coloring and Breaking Cycles

To systematically eliminate monochromatic cycles, we classify the A-nodes (edges of G) based on their configuration in A_1:

- Let Z be the set of all A-nodes belonging to A-cycles in A_1.
- Let P be the set of all A-nodes belonging to A-paths that contain at least one A-edge (i.e., non-isolated A-nodes not in Z).
- Let S be the set of all isolated A-nodes in A_1.

Furthermore, for any A-path, we define its first and last A-nodes as *A-end nodes*. We first restrict our focus to the subgraph $G' = (V, Z \cup P)$. Our objective is to ensure that G' can be 2-colored such that each color class contains strictly node-disjoint paths. Because G lacks 2-cycles, any monochromatic cycle formed in G' must consist of at least three edges. More importantly, due to the established alternating coloring, the only way a monochromatic cycle can manifest in G' is if all participating edges are A-end nodes of various A-paths in P. Furthermore, degree restrictions and the fact that every A-path in P represents at least two edges in G guarantee that any such monochromatic cycles in G' are entirely node-disjoint.

To destroy a monochromatic cycle containing edges $e_1, e_2, \ldots, e_l$ in G', we forcefully introduce a color conflict between two adjacent edges. We arbitrarily select e_1 and e_2 and add a new auxiliary edge $\{e_1, e_2\}$ to our conflict set. Applying this to all monochromatic cycles in G' produces an augmented edge set F_2, giving a new auxiliary graph $A_2 = (Z \cup P, F_2)$.

Since the problematic cycles in G' are node-disjoint, there are at most $|E|/3$ such cycles, meaning they can be identified and neutralized in polynomial time. While introducing the new A-edges in F_2 may fuse some A-paths into new A-cycles, these newly formed A-cycles maintain an even length. This is because adding an A-edge $\{e_1, e_2\}$ structurally represents the meeting of two heads or two tails, preserving the alternating parity required for 2-colorability.

Lemma 17. *The edges of the subgraph $G' = (V, Z \cup P)$ can be 2-colored such that each color class consists strictly of node-disjoint paths. This refined coloring is computable in polynomial time.*

Following this augmentation, we update the sets Z and P to reflect the transfer of A-nodes from paths into the newly formed A-cycles of A_2. The final step of the algorithm must then correctly integrate the isolated edges of set S without closing any new monochromatic cycles.

4.2.3 Phase 3: Integrating Isolated Edges

The Special Case Assume temporarily that any directed cycle in G contains at least two edges belonging to S. Under this assumption, we construct a new auxiliary directed multigraph, B, designed to track the monochromatic paths already established in the refined subgraph $G' = (V, Z \cup P)$:

- **Vertices:** The vertices of B correspond directly to the edges in S.
- **Edges:** A directed A-edge (e, f) is added to B if there exists a monochromatic path in G' connecting the head of e to the tail of f. This new A-edge inherits the color of that connecting path.
- **Adjacency Handling:** If two edges in S are directly adjacent in G (e.g., $e = (u, v)$ and $f = (v, w)$), two parallel A-edges are added between them in B, one for each color.

Because of our temporary assumption that every cycle in G contains at least two edges from S, the multigraph B is guaranteed to be free of self-loops. Furthermore, degree constraints dictate that every vertex in B has an indegree and outdegree of at most 2, and any incoming or outgoing pairs of A-edges must be assigned different colors.

Lemma 18. *If the vertices of B can be 2-colored such that every monochromatic cycle in B contains at least one vertex of the opposite color, then the entirety of G can be validly 2-colored into node-disjoint paths.*

To deterministically find the coloring required by Lemma 18, we analyze the monochromatic cycles of B by constructing a secondary undirected multigraph, B^*. The vertices of B^* represent the monochromatic cycles of B, and an edge exists between two vertices in B^* if their corresponding cycles share a vertex in B.

The coloring algorithm for B proceeds recursively based on the concept of *free A-nodes* (vertices in B that belong to at most one monochromatic cycle):

1. If a connected component in B contains a free A-node, we assign it the color opposite to its cycle, virtually destroying that cycle. This removal breaks the component into smaller subcomponents, each guaranteed to now possess a free A-node, allowing the process to recurs.

2. If a component lacks a free A-node, its corresponding structure in B^* must contain a cycle or a double edge. Because all cycles in B^* have even length, we can alternatively 2-color the elements of the B^* cycle. This simultaneous color assignment shatters all associated cycles in B, generating free A-nodes and returning us to the first case.

The General Case: Inductive Graph Reduction To remove the restrictive assumption that every cycle in G contains at least two edges from S, the algorithm uses induction on the number of vertices, $|V|$.

First, trivially isolated edges in S (those sharing no endpoints with any other edge in E) are colored arbitrarily, and adjacent pairs in S are assigned opposite colors and removed from consideration. Let $T \subseteq S$ be the remaining subset of these isolated A-nodes.

For each edge $t \in T$, we determine if its tail is reachable from its head within the currently colored subgraph $(V, Z \cup P)$. If no such path exists for any t, then any cycle formed by incorporating T must necessarily contain at least two edges from T. This directly satisfies our earlier assumption, allowing us to safely apply the coloring logic of multigraph B.

However, if there exists a cycle containing exactly one edge $e \in T$, the algorithm executes a structural graph contraction. Because G contains no 2-cycles, e must be adjacent to an edge f that is an endpoint of an established path. By strategically discarding e, f, and their shared junction, and rewiring the surrounding edges to bypass the removed segment, we produce a strictly smaller graph $\hat{G}$. This smaller graph inherits the necessary structural properties, allowing the path-coloring to proceed inductively.

Inductive Step and Subcase Analysis Throughout this induction, the two colors are denoted 0 and 1, so that for a color $C \in \{0, 1\}$ the opposite color is $1 - C$.

The discard-and-rewire step splits into several subcases, all worked out by Bläser [Blä04] except one. Below we number them Case 1 through Case 3: Cases 1 and 2 are representative examples of case types Bläser already covers in full (the remaining case types follow the same pattern and are left to [Blä04]), while Case 3 is the one genuinely missing from his figures.

Figure 4.1 shows the local picture we start from: the edge $e = (x, y)$ we are trying to remove, its preceding path-edge f_1, the edge continuing the cycle at y (call it g_1), and the

other edge at x, a_1, whose far endpoint we write u. The other edge at y is c_1, with far endpoint q.

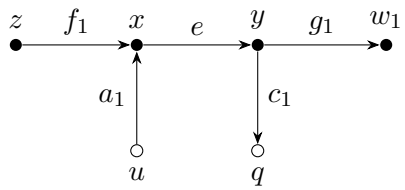

Figure 4.1: Local picture around e, with all four surrounding edges f_1, e, a_1, c_1 shown, before anything is discarded.

Discarding e, f_1 removes x and z from consideration and rewires a_1 to end at y instead of x, giving Figure 4.2. Everything that follows is based on comparing u (where a_1 now starts) with q (where c_1 ends).

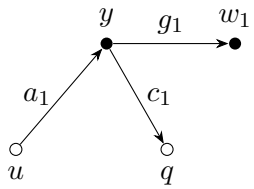

Figure 4.2: After discarding e, f_1 and rewiring a_1. Whether $u = q$ decides the case.

Case 1 ($u \neq q$): nothing else collides. No 2-cycle is created, so we recurse on Figure 4.2 directly, with a_1 ending at y as drawn. Let C be the color a_1 receives from the recursion. We then set $e = C$ and $f_1 = 1 - C$ (Figure 4.3).

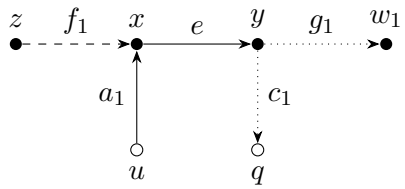

Figure 4.3: Recovered coloring for $u \neq q$, undoing the reduction in full back to Figure 4.1: a_1, e share color C (solid) and f_1 gets $1 - C$ (dashed), matching `path_coloring.py`. g_1, c_1 (dotted) already have real colors too, assigned by this same recursive call – just not ones this rule determines, so we leave them unstyled.

Case 2 ($u = q = v$, and v is the tail of the third edge h at v) Beyond a_1, c_1 colliding, nothing else needs to be touched.

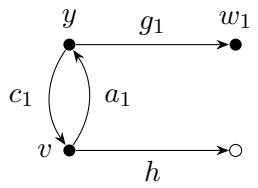

Figure 4.4: Setup: a_1, c_1 close a 2-cycle at v, while h leaves v toward an unrelated node.

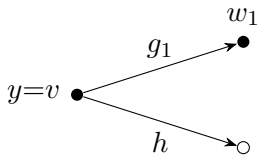

Figure 4.5: Discard a_1, c_1 and identify y with v. Recurse on the smaller graph.

Undoing this reduction in full means restoring not just a_1, c_1 but also x, z with e, f_1. Crucially, a_1 reverts to its original form (v, x) – the rewired (v, y) version existed only inside the reduction. We set a_1 to g_1's color, c_1 to h's color, then e to a_1's color and f_1 to the opposite.

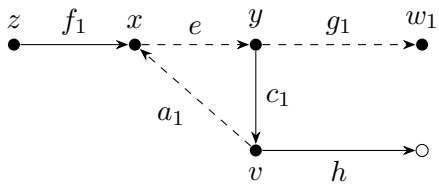

Figure 4.6: Recovered coloring, undone in full: g_1, a_1, e share one color (dashed), h, c_1, f_1 the other (solid). The dashed class is the path $v \to x \to y \to w_1$ and the solid class is $z \to x$ together with $y \to v \to p$ – both simple, no cycle.

Case 3 ($u = q = v$, and v is the head of h): omitted from [Blä04]. Write $w_1 = \mathit{head}(g_1)$ and $w_2 = \mathit{tail}(h)$. If $w_1 = w_2$ this is a loop, not a fresh 2-cycle, fixed by discarding g_1, c_1, h, e, f_1 and recoloring from the next chain edge at w_1. Assume $w_1 \neq w_2$. Now w_1 carries a second edge c_2 (its chain neighbor g_2 leads elsewhere, dashed below) – and it matters whether c_2 happens to land exactly on w_2.

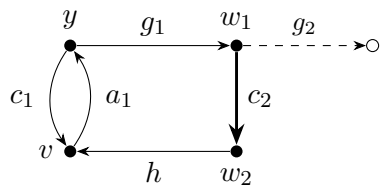

Figure 4.7: Setup: a_1, c_1 close a 2-cycle at v, and w_1's loose edge c_2 (bold) happens to coincide with $w_2 = \mathit{tail}(h)$ – unrelated to where the chain edge g_2 (dashed) actually goes.

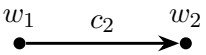

Figure 4.8: Discard g_1, h, c_1, a_1 and the nodes x, y, v, leaving c_2 untouched. Recurse on what remains.

Let v_e be the third edge at w_2 (other than h, c_2), colored C by the recursion. We set

$$f_1 = C, \quad e = C, \quad c_1 = C, \qquad g_1 = 1 - C, \quad h = 1 - C, \quad a_1 = 1 - C, \tag{4.1}$$

and leave c_2 with whatever color the recursion already gave it.

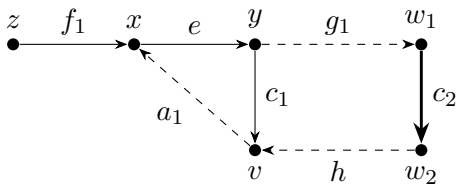

Figure 4.9: Recovered coloring, undone in full back to z, x, with a_1 reverted to (v, x): f_1, e, c_1 (color C, solid) versus g_1, a_1, h (color $1 - C$, dashed). The edge c_2 (bold) keeps whatever color the recursion assigned it.

Bläser's own Fig. A.10 in [Blä04] handles a different coincidence: the chain edge g_2 (not c_2) leading to a node whose *own* further edge (his c_3) coincides with h, resolved by discarding g_2, g_1, h, c_1, a_1 and splicing a further chain edge g_3 into w_1. The case above – c_2 itself landing on w_2 – is not among his figures. It is, however, exactly the branch `two_cycle_edge == c_2` in the reference implementation (`path_coloring.py`, function `_lemma_15_inductive_coloring`), which is why we have spelled it out in full.

4.3 Complexity and Running Time

The efficiency of this coloring scheme relies fundamentally on the graph's degree restrictions. Since every vertex has a total degree of at most 3, the number of edges is strictly bounded by $|E| \leq \frac{3}{2}|V|$. The inductive reduction eliminates at least one vertex per step, so the reduction is applied at most $|V|$ times. Each individual step, however, does not run in $O(|E|)$ time: constructing the auxiliary graph A_1 requires comparing every pair of edges for a shared endpoint, which takes $O(|E|^2) = O(|V|^2)$ time, and locating the target edge e requires testing, for each of the $O(|V|)$ candidates in T, whether it lies on a cycle of G', at a further cost of $O(|V| + |E|) = O(|V|)$ per candidate – again $O(|V|^2)$ in total. Hence each inductive step costs $O(|V|^2)$ time, and since at most $|V|$ steps are performed, the entire path-coloring algorithm executes deterministically in $O(|V|) \cdot O(|V|^2) = O(n^3)$ time.

Chapter 5

Simulations and results

This chapter compares the algorithms implemented in this work against the other Shortest Common Superstring heuristics and approximation algorithms already present in the repository[1]: the classical Greedy (maximum overlap) heuristic with its MGreedy and TGreedy variants, Group Merge, the cycle-cover algorithms of Teng and Yao [TY97] and of Paluch, Elbassioni and van Zuylen [PEZ12], and the $\frac{38}{63}$-approximation directed Max-TSP algorithm of Kosaraju, Park and Stein [KPS94] used as an overlap approximation subroutine.

Four of the nine implementations benchmarked here were contributed as part of this work[2]: the two Breslauer–Jiang–Jiang algorithms of Chapter 2, the Kosaraju–Park–Stein Max-TSP approximation of Chapter 3 that the second of them uses as a subroutine, and the path-coloring algorithm of Chapter 4 that Version 2 of Kosaraju–Park–Stein requires in turn. Table 5.1 lists where each of them is implemented, with the contributions of this work set in bold.

Table 5.1: Where each benchmarked algorithm is implemented, as a path relative to the root of the repository. The implementations contributed by this work are set in **bold**.

Algorithm	**File**
Greedy, MGreedy, TGreedy, Group Merge, brute-force optimum	`shortest_common_superstring/shortest_common_superstring.py`
Teng–Yao	`shortest_common_superstring/teng_yao.py`
Paluch–Elbassioni–van Zuylen	`shortest_common_superstring/paluch_elbassoni_zuylen.py`
Kosaraju–Park–Stein ($\frac{38}{63}$)	`shortest_common_superstring/kosaraju.py`
Path-coloring lemma	`shortest_common_superstring/path_coloring.py`
Breslauer–Jiang–Jiang $2\frac{2}{3}$ and $2\frac{25}{42}$	`shortest_common_superstring/breslauer.py`

[1] `https://github.com/krzysztof-turowski/string-algorithms`
[2] `https://github.com/krzysztof-turowski/string-algorithms/pull/61`

5.1 Method

No standard public benchmark suite exists for the Shortest Common Superstring problem, unlike, e.g., TSPLIB for the Traveling Salesman Problem. Three sources of test instances were therefore used.

- **Random strings.** Independent words over an alphabet of controlled size, with lengths drawn uniformly from 8 to 15.
- **Natural text.** Fragments of length 25 cut at random positions from *Pan Tadeusz*, lowercased and stripped of punctuation.
- **DNA.** Fragments of length 40 cut at random positions from the genome of bacteriophage phiX174 [San+77], NCBI accession NC_001422.1 [Nat26a].

The last two follow the random-fragment methodology used in the DNA assembly literature: a single longer source string is cut into overlapping pieces at random positions, rather than generating fully independent random words.

Because the true optimum is computable by brute force only for very small instances, each result is reported relative to the shortest superstring found by any of the algorithms on the same instance. A value of 1.0000 therefore means the algorithm was never beaten on any instance of that experiment. Section 5.5 covers instances small enough for the brute-force solver and compares against the true optimum instead.

Every reported value is a mean over independent repetitions of the same experiment. Running times were measured on a single core with the overlap matrix computed inside the timed region, so they include the cost of building the overlap graph.

5.2 Overview

Table 5.2 pools the four comparative experiments of the following sections into one ranking, over all instance sizes, alphabets and sources at once. The experiment on tiny instances is excluded from it, because there the reference is the true optimum rather than the best result found.

Table 5.2: Overview of all benchmarked algorithms over the four comparative experiments: superstring length relative to the best result found on the same instance, and mean running time.

Algorithm	**Length / best found**		**Mean time**
	mean	worst	(ms)
Greedy (max overlap)	1.0052	1.0364	28.42
MGreedy	1.0115	1.0992	28.17
TGreedy	1.0071	1.0577	29.77
Group Merge	1.0093	1.0534	6.60
Teng–Yao	1.0191	1.1264	5.58
Paluch–Elbassioni–van Zuylen	1.0018	1.0221	3.42
Kosaraju–Park–Stein ($\frac{38}{63}$)	1.0046	1.1074	17.31
Breslauer–Jiang–Jiang $2\frac{2}{3}$	1.0794	1.3158	2.99
Breslauer–Jiang–Jiang $2\frac{25}{42}$	1.0772	1.3158	3.03

5.3 Scaling with instance size and alphabet

Figure 5.1 varies the number of input strings at a fixed alphabet $\{a, b, c, d\}$, and Figure 5.2 varies the alphabet size at a fixed $n = 20$. Figure 5.3 reports the running times measured on the instances of Figure 5.1.

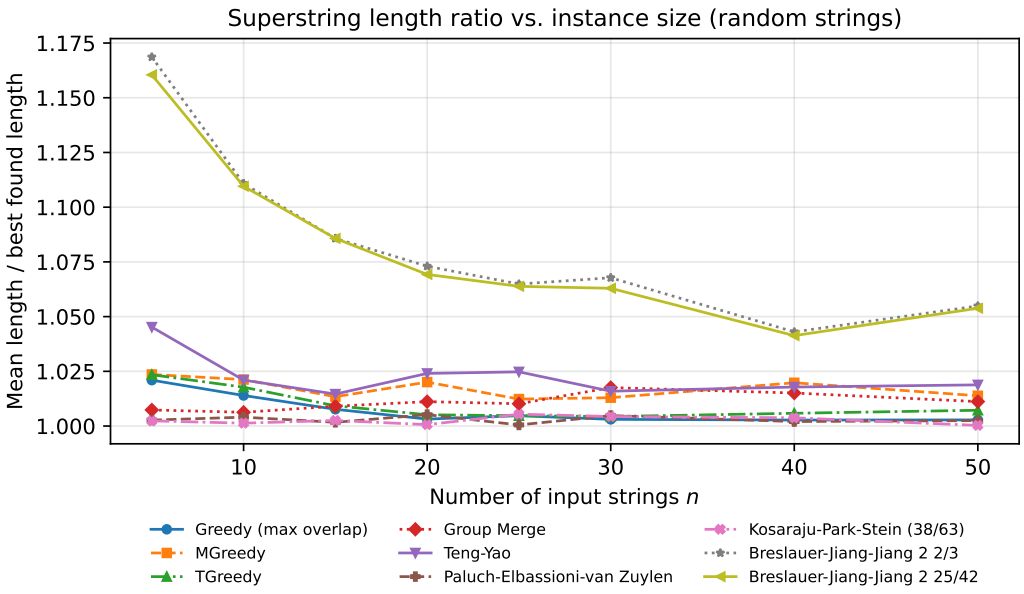

Figure 5.1: Mean superstring length relative to the best result found, as a function of the number of input strings n (random strings, alphabet $\{a, b, c, d\}$, word lengths 8–15, 8 repeats per point).

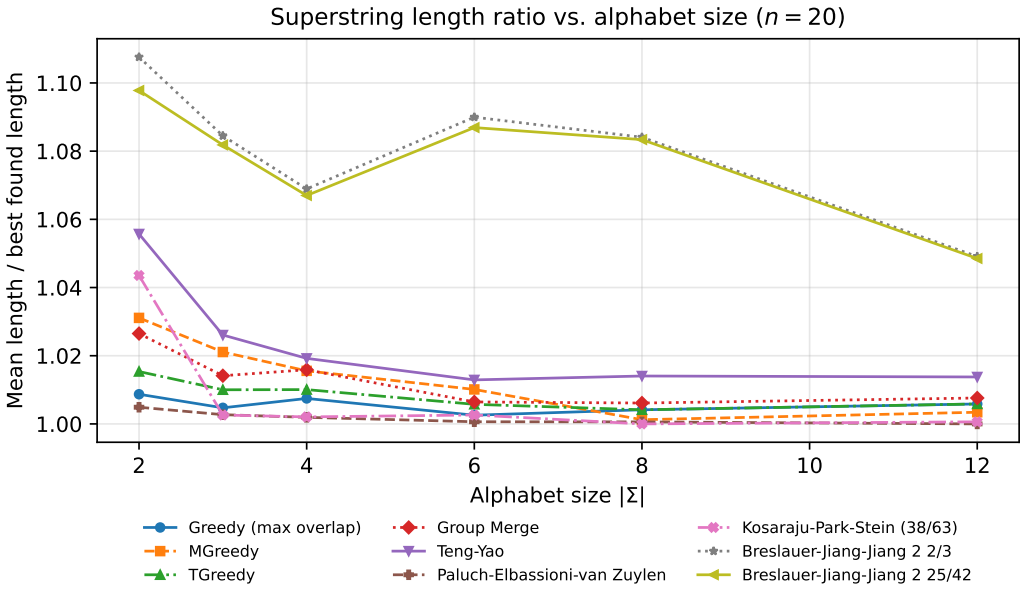

Figure 5.2: Mean superstring length relative to the best result found, as a function of alphabet size (random strings, $n = 20$, word lengths 8–15, 8 repeats per point).

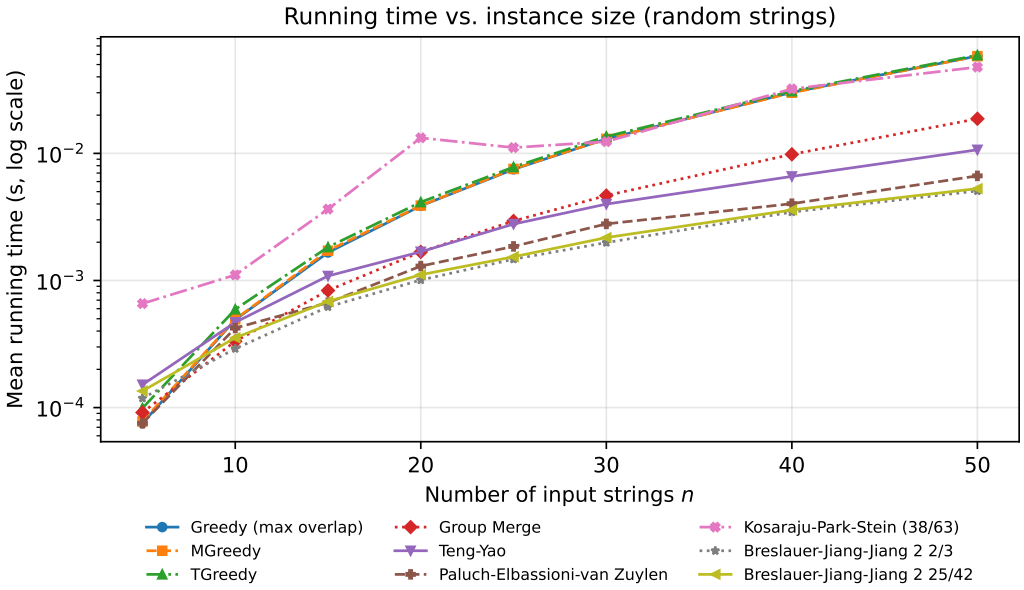

Figure 5.3: Mean running time (log scale) as a function of the number of input strings n, same instances as Figure 5.1.

5.4 Real-world data: natural text and DNA

Figure 5.4 puts the three instance sources side by side at the same relative scale. Table 5.3 reports absolute lengths on the DNA instances rather than ratios, together with the compression achieved relative to the plain concatenation of all reads.

Table 5.3: Mean superstring length on real DNA fragments (bacteriophage phiX174, NCBI NC_001422.1 [Nat26a]), averaged over $k \in \{8, 15, 25, 40, 60\}$ reads of length 40 at 5 repeats each. The last column is the length divided by the length of the plain concatenation of all reads, i.e. the compression actually achieved.

Algorithm	**Mean superstring length**	**vs. concatenation**
Greedy (max overlap)	986.1	0.833
MGreedy	988.0	0.834
TGreedy	986.8	0.833
Group Merge	991.4	0.837
Teng–Yao	998.5	0.843
Paluch–Elbassioni–van Zuylen	985.4	0.832
Kosaraju–Park–Stein ($\frac{38}{63}$)	994.4	0.840
Breslauer–Jiang–Jiang $2\frac{2}{3}$	1044.9	0.883
Breslauer–Jiang–Jiang $2\frac{25}{42}$	1044.6	0.882
Plain concatenation (no overlap)	1184.0	1.000

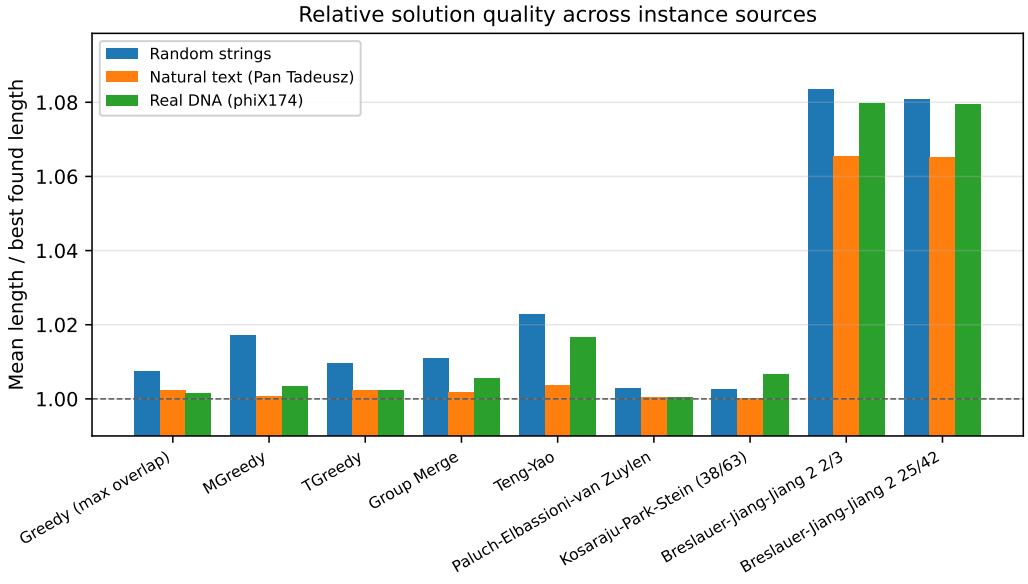

Figure 5.4: Mean superstring length relative to the best result found, grouped by instance source: random strings, fragments of a natural-language text (*Pan Tadeusz*), and fragments of a real genome (bacteriophage phiX174).

5.5 Comparison against the true optimum

The brute-force solver implemented alongside these algorithms enumerates candidate superstrings by increasing length, which is tractable only on very small inputs. Table 5.4 therefore uses 20 independent instances of three or four words of length three or four over a binary alphabet, on which the solver still terminates quickly, and reports every algorithm against the true optimum rather than against the best result found.

Table 5.4: Comparison against the true optimum, computed by brute-force enumeration. A single instance consists of $n \in \{3, 4\}$ words of length 3 or 4 over the alphabet $\{a, b\}$, and the table averages over 20 such instances.

Algorithm	**Length / OPT**		**Optimal**
	mean	worst	instances
Greedy (max overlap)	1.1541	1.2500	0/20
MGreedy	1.1091	1.5000	11/20
TGreedy	1.1541	1.2500	0/20
Group Merge	1.0000	1.0000	20/20
Teng–Yao	1.0872	1.3333	11/20
Paluch–Elbassioni–van Zuylen	1.0083	1.1667	19/20
Kosaraju–Park–Stein ($\frac{38}{63}$)	1.0674	1.3750	14/20
Breslauer–Jiang–Jiang $2\frac{2}{3}$	1.1488	1.4286	9/20
Breslauer–Jiang–Jiang $2\frac{25}{42}$	1.1488	1.4286	9/20

5.6 The cost of a single cycle-breaking version

The Kosaraju–Park–Stein algorithm of Chapter 3 computes one maximum weight cycle cover and then breaks it into paths in three independent ways. Version 1 discards the lightest edge of every cycle (Section 3.2), Version 2 contracts the 2-cycles and applies the path-coloring lemma of Chapter 4 (Section 3.3), and Version 3 cuts long cycles into pieces and recombines them through a maximum weight matching (Section 3.4). The algorithm returns the heaviest of the three tours, and the $\frac{38}{63}$ guarantee of Theorem 12 is a statement about that maximum.

To measure what each version contributes on its own, the same algorithm was run four times per instance with the version set restricted to $\{1\}$, $\{2\}$, $\{3\}$ and $\{1, 2, 3\}$. All four start from the identical cycle cover, so every difference comes from the cycle-breaking step alone. Lengths in Table 5.5 and Figure 5.5 are relative to the best of these four variants, not to the full set of algorithms benchmarked earlier.

Table 5.5: Effect of restricting the Kosaraju–Park–Stein algorithm to a single cycle-breaking version instead of taking the heaviest of the three. Lengths are relative to the best of these four variants on the same instance. The random columns cover $n \in \{5, \ldots, 50\}$ (8 repeats per point), the DNA column covers phiX174 reads with $k \in \{8, 15, 25, 40\}$ (5 repeats per point).

Variant	**Random strings**		**DNA**	**Shortest**	**Mean time**
	mean	worst	mean	found	(ms)
Version 1 only	1.0006	1.0128	1.0004	76/84	1.38
Version 2 only	1.0365	1.0773	1.0284	7/84	4.94
Version 3 only	1.0043	1.0213	1.0009	48/84	11.25
Best of 1–3 (full algorithm)	1.0000	1.0000	1.0000	84/84	14.81

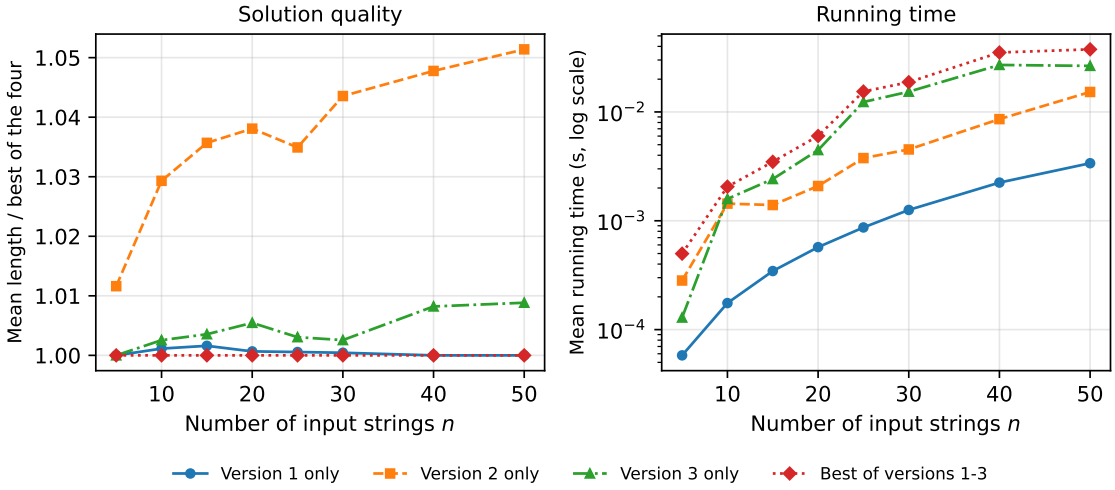

Figure 5.5: Quality and running time of the Kosaraju–Park–Stein algorithm when restricted to a single cycle-breaking version, as a function of the number of input strings n (random strings, alphabet $\{a, b, c, d\}$, word lengths 8–15, 8 repeats per point). Lengths are relative to the best of the four variants on the same instance.

5.7 Large instances

The experiments above stay below $n = 60$, where every algorithm finishes in milliseconds and the running times compare implementation constants rather than asymptotic behaviour. This section repeats the comparison on instances large enough for the cubic step to dominate. Fragments were cut from *Pan Tadeusz* at $n \in \{200, 400, 800, 1600\}$ and from the genome of phage lambda [San+82], NCBI accession NC_001416.1 [Nat26b], at $n \in \{200, 400, 800\}$ with reads of length 100. Each point is a single instance handed to all nine algorithms, and each run was given a 90-minute limit, which none of them reached.

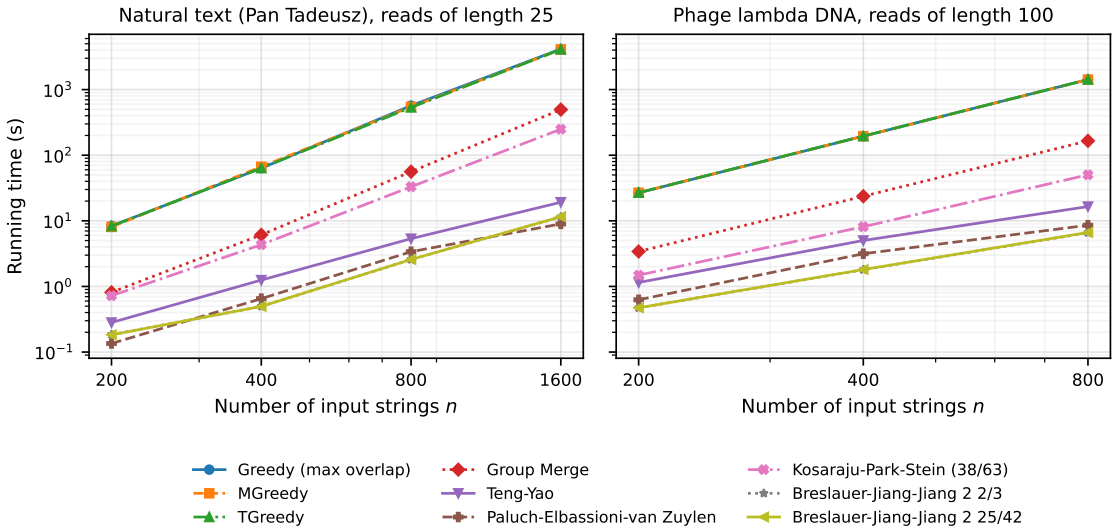

Figure 5.6: Running time on large instances, both axes logarithmic, so the slope of each line is the empirical exponent. Fitted on the text instances, the three greedy variants and Group Merge follow slopes of 3.0 and 3.1, Teng–Yao, Paluch–Elbassioni–van Zuylen and the two Breslauer–Jiang–Jiang algorithms fall between 2.0 and 2.1, and Kosaraju–Park–Stein sits between the two groups at 2.8. Greedy, MGreedy and TGreedy differ by at most 6 percent at any point and are drawn on top of one another.

Table 5.6: The largest instance measured for each source: superstring length relative to the shortest found on that instance, and running time. Every algorithm was given the same instance.

Algorithm	**Text,** $n = 1600$		**DNA,** $n = 800$	
	length / best	time (s)	length / best	time (s)
Greedy (max overlap)	1.0000	4152.5	1.0000	1421.3
MGreedy	1.0004	4091.3	1.0003	1420.8
TGreedy	1.0002	4090.5	1.0001	1422.1
Group Merge	1.0054	495.7	1.0507	165.5
Teng–Yao	1.0005	19.2	1.0034	16.5
Paluch–Elbassioni–van Zuylen	1.0000	9.0	1.0000	8.6
Kosaraju–Park–Stein ($\frac{38}{63}$)	1.0033	249.2	1.0248	50.2
Breslauer–Jiang–Jiang $2\frac{2}{3}$	1.0007	11.5	1.0123	6.6
Breslauer–Jiang–Jiang $2\frac{25}{42}$	1.0007	11.5	1.0122	6.6

5.8 Conclusions

Read on the small instances alone, the comparison says that the choice of algorithm hardly matters. Across the four experiments of Sections 5.5 and earlier, every method except the two Breslauer–Jiang–Jiang variants stays within roughly two percent of the best result found, and the ordering by measured quality bears no relation to the ordering by provable constant. Paluch–Elbassioni–van Zuylen leads at 1.0018, but Greedy reaches 1.0052 with no approximation machinery at all, and the gap between them costs a few tens of milliseconds either way.

Section 5.7 shows that this reading is an artefact of scale. On $n = 1600$ fragments of natural text the two algorithms still return effectively the same string, differing by 1 character, but Greedy now needs 69 minutes against 9.0 seconds for Paluch–Elbassioni–van Zuylen, a factor of 463. The empirical exponents behind that gap are 3.0 and roughly 2.0, so it widens with every doubling of n. Which algorithm to use is therefore settled on running time rather than on length, and it is a difference the small instances cannot show, because there the same gap is a few tens of milliseconds.

The same correction applies to the algorithms of Chapter 2, which look worst in the chapter and are not. They average 1.0772 on the small instances and 1.3158 in the worst case measured, far behind everything else, because their Step 3 chooses cycle representatives t_C so that the subsequent bound can be proved, and that choice gives up overlap a greedy merge would have taken. The overlap given up is fixed per cycle, so it dominates when there are few cycles and amortises when there are many. At $n = 1600$ the same two algorithms come in at 1.0007, within a thousandth of the best result, in under twelve seconds. Their weakness was a property of the instance sizes, not of the construction.

Inside Kosaraju–Park–Stein the picture is different and does not improve with scale. Section 5.6 shows that Version 1, the one-line rule discarding the lightest edge of every cycle, is 0.06 % off the full algorithm while running 13 times faster, so the path-coloring construction of Chapter 4 and the piece-recombination of Version 3 contribute almost nothing to the measured output. These two versions exist to hold up the $\frac{38}{63}$ bound, not to shorten strings.

Was implementing these algorithms worth it? On the evidence collected here, yes, though not for the reason the approximation ratios suggest. None of them returns a noticeably shorter superstring than a maximum-overlap greedy merge, and on small inputs none of them needs to. What they provide is a construction that stays affordable when the input grows, which is exactly where the greedy heuristic stops being usable, and a guarantee that holds on inputs no benchmark can be asked to generate. The measurements add the part the proofs leave open, namely how much of the worst-case pessimism survives contact with realistic data. Very little of it does, and the practical argument for these algorithms turns out to rest on their running time rather than on the bounds they were designed to establish.

Bibliography

[Kuh55] Harold W. Kuhn. "The Hungarian Method for the Assignment Problem". In: *Naval Research Logistics Quarterly* 2.1-2 (1955), pp. 83–97. DOI: 10.1002/nav.3800020109.

[LS62] Roger C. Lyndon and Marcel P. Schützenberger. "The Equation $a^M = b^N c^P$ in a Free Group". In: *Michigan Mathematical Journal* 9.4 (1962), pp. 289–298. DOI: 10.1307/mmj/1028998766.

[Viz64] Vadim G. Vizing. "On an Estimate of the Chromatic Class of a p-Graph". In: *Diskret. Analiz* 3 (1964). In Russian, pp. 25–30.

[Edm65a] Jack Edmonds. "Maximum Matching and a Polyhedron with 0,1-Vertices". In: *Journal of Research of the National Bureau of Standards Section B* 69B (1965), pp. 125–130. DOI: 10.6028/jres.069B.013.

[Edm65b] Jack Edmonds. "Paths, Trees, and Flowers". In: *Canadian Journal of Mathematics* 17 (1965), pp. 449–467. DOI: 10.4153/CJM-1965-045-4.

[Tom71] N. Tomizawa. "On Some Techniques Useful for Solution of Transportation Network Problems". In: *Networks* 1.2 (1971), pp. 173–194. DOI: 10.1002/net.3230010206.

[EK72] Jack Edmonds and Richard M. Karp. "Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems". In: *Journal of the ACM* 19.2 (1972), pp. 248–264. DOI: 10.1145/321694.321699.

[Kar72] Richard M. Karp. "Reducibility Among Combinatorial Problems". In: *Complexity of Computer Computations*. Ed. by Raymond E. Miller and James W. Thatcher. New York, NY, USA: Plenum Press, 1972, pp. 85–103. DOI: 10.1007/978-1-4684-2001-2_9.

[SG76] Sartaj Sahni and Teofilo Gonzalez. "P-Complete Approximation Problems". In: *Journal of the ACM* 23.3 (1976), pp. 555–565. DOI: 10.1145/321958.321975.

[San+77] Frederick Sanger et al. "Nucleotide Sequence of Bacteriophage ϕX174 DNA". In: *Nature* 265.5596 (1977), pp. 687–695. DOI: 10.1038/265687a0.

[CV78] Yves Césari and Michel Vincent. "Une caractérisation des mots périodiques". In: *Comptes Rendus Hebdomadaires des Séances de l'Académie des Sciences* 286.A (1978), pp. 1175–1177.

[GJ79] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[GMS80] John Gallant, David Maier, and James A Storer. "On finding minimal length superstrings". In: *Journal of Computer and System Sciences* 20.1 (1980), pp. 50–58. DOI: 10.1016/0022-0000(80)90004-5.

[San+82] Frederick Sanger et al. "Nucleotide Sequence of Bacteriophage λ DNA". In: *Journal of Molecular Biology* 162.4 (1982), pp. 729–773. DOI: 10.1016/0022-2836(82)90546-0.

[SS82] James A Storer and Thomas G Szymanski. "Data compression via textual substitution". In: *Journal of the ACM (JACM)* 29.4 (1982), pp. 928–951. DOI: 10.1145/322344.322346.

[Lot83] M. Lothaire. *Combinatorics on Words*. Reading, MA, USA: Addison-Wesley, 1983.

[PSU84] Hannu Peltola, Hans Söderlund, and Esko Ukkonen. "SEQAID: a DNA sequence assembling program based on a mathematical model". In: *Nucleic Acids Research* 12.1Part1 (1984), pp. 307–321. DOI: 10.1093/nar/12.1Part1.307.

[Law+85] Eugene L Lawler et al., eds. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Chichester, UK: John Wiley & Sons, 1985.

[CP91] Maxime Crochemore and Dominique Perrin. "Two-Way String-Matching". In: *Journal of the ACM* 38.3 (1991), pp. 650–674. DOI: 10.1145/116825.116845.

[Blu+94] Avrim Blum et al. "Linear approximation of shortest superstrings". In: *Journal of the ACM (JACM)* 41.4 (1994), pp. 630–647. DOI: 10.1145/179812.179818.

[CR94] Maxime Crochemore and Wojciech Rytter. *Text Algorithms*. New York, NY, USA: Oxford University Press, 1994.

[KPS94] S. Rao Kosaraju, James K. Park, and Clifford Stein. "Long Tours and Short Superstrings". In: *Proceedings of the 35th Annual IEEE Symposium on Foundations of Computer Science*. IEEE, 1994, pp. 166–177. DOI: 10.1109/SFCS.1994.365696.

[BJJ97] Dany Breslauer, Tao Jiang, and Zhigen Jiang. "Rotations of Periodic Strings and Short Superstrings". In: *Journal of Algorithms* 24.2 (1997). Preliminary version: BRICS Report Series RS-96-21, June 1996, pp. 340–353. DOI: 10.1006/jagm.1997.0861.

[Czu+97] Artur Czumaj et al. "Sequential and parallel approximation of shortest superstrings". In: *Journal of Algorithms* 23.1 (1997), pp. 74–100. DOI: 10.1006/jagm.1996.0823.

[SM97] João Carlos Setubal and João Meidanis. *Introduction to Computational Molecular Biology*. Boston, MA, USA: PWS Publishing Company, 1997.

[TY97] Shang-Hua Teng and F Frances Yao. "Approximating shortest superstrings". In: *SIAM Journal on Computing* 26.2 (1997), pp. 410–417. DOI: 10.1137/S0097539794286125.

[AS98] Chris Armen and Clifford Stein. "A 2 2/3 superstring approximation algorithm". In: *Discrete Applied Mathematics* 88.1-3 (1998), pp. 29–57. DOI: 10.1016/S0166-218X(98)00065-1.

[Vaz01] Vijay V. Vazirani. *Approximation Algorithms*. Berlin, Heidelberg: Springer, 2001.

[Blä04] Markus Bläser. "An 8/13-approximation algorithm for the asymmetric maximum TSP". In: *Journal of Algorithms* 50.1 (2004), pp. 23–48. DOI: 10.1016/S0196-6774(03)00112-3.

[Kap+05] Haim Kaplan et al. "Approximation algorithms for asymmetric TSP by decomposing directed regular multigraphs". In: *Journal of the ACM (JACM)* 52.4 (2005), pp. 602–626. DOI: 10.1145/1082036.1082041.

[PEZ12] Katarzyna Paluch, Khaled Elbassioni, and Anke van Zuylen. "Simpler Approximation of the Maximum Asymmetric Traveling Salesman Problem". In: *29th International Symposium on Theoretical Aspects of Computer Science (STACS 2012)*. Vol. 14. LIPIcs. 2012, pp. 501–506. DOI: 10.4230/LIPIcs.STACS.2012.501.

[Muc13] Marcin Mucha. "Lyndon words and short superstrings". In: *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM. 2013, pp. 958–972. DOI: 10.1137/1.9781611973105.69.

[Pal20] Katarzyna Paluch. *New Approximation Algorithms for Maximum Asymmetric Traveling Salesman and Shortest Superstring*. 2020. arXiv: 2005.10800.

[EMV23] Matthias Englert, Nicolaos Matsakis, and Pavel Veselý. "Approximation Guarantees for Shortest Superstrings: Simpler and Better". In: *34th International Symposium on Algorithms and Computation (ISAAC 2023)*. Vol. 283. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 29:1–29:17. DOI: 10.4230/LIPIcs.ISAAC.2023.29.

[Nat26a] National Center for Biotechnology Information. *Bacteriophage phiX174, Complete Genome*. https://www.ncbi.nlm.nih.gov/nuccore/NC_001422.1. NCBI Reference Sequence NC_001422.1, accessed 26 August 2026. 2026.

[Nat26b] National Center for Biotechnology Information. *Enterobacteria Phage Lambda, Complete Genome*. https://www.ncbi.nlm.nih.gov/nuccore/NC_001416.1. NCBI Reference Sequence NC_001416.1, accessed 30 August 2026. 2026.